Expert Systems With Applications 290 (2025) 128258

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Review

Metaheuristic approaches for the stochastic capacitated multi-depot vehicle routing problem with pickup and delivery

Brenner Humberto Ojeda Rios [a,b,*], Eduardo Candido Xavier [b]

[a] Universidad de Ingeniería y Tecnología (UTEC), Jr. Medrano Silva 165, Barranco, Lima, Peru
[b] State University of Campinas, Cidade Universitária Zeferino Vaz - Barão Geraldo, Campinas - SP, 13083-970, Brasil, Campinas, São Paulo, Brazil

ARTICLE INFO

Keywords:
Stochastic vehicle routing
Transportation
Metaheuristic

ABSTRACT

The classical Vehicle Routing Problem (VRP) is to determine optimal routes for m identical vehicles that must visit a set of customers, departing from and returning to a depot. The following restrictions must be satisfied: (1) each vehicle begins and ends at the depot and (2) each customer is visited exactly once. We consider a novel variant of the VRP that incorporates multiple depots, vehicle capacity and stochastic information about future incoming of customers. We denote this problem as the stochastic capacitated multi-depot VRP with pickup and delivery (SCMVRPPD). The SCMVRPPD is an extension of the capacitated VRP with pickup and delivery, in which there is a different depot for each vehicle and the pickup and delivery points (each pair associated with a customer) are stochastic. The problem is to compute routes visiting all pairs of pickup and delivery minimizing the expected total cost of routes, since the real occurrence of each pair of pickup/delivery is random. In this case, some of the a priori designed routes can fail. We propose strategies to correct routes with failures.

For each different strategy, we propose methods to efficiently compute the expected cost of a solution and, for one particular correction strategy, we propose a closed-form expression. We propose three algorithms to tackle the problem: a Tabu search (TS) heuristic, a Generalized Variable Neighborhood Search (GVNS) and an Iterated Local Search (ILS-VND) method. We used modified instances from a well-known benchmark of instances of the VRP to compare the algorithms. Computational results clearly show the superiority of the proposed ILS heuristic over alternative solution approaches.

1. Introduction

The Vehicle Routing Problem (VRP), originally introduced by Dantzig and Ramser (1959), is a widely studied network optimization challenge that involves serving a set of geographically dispersed customers using a fleet of vehicles. In this problem, each vehicle must begin and conclude its route at a central depot. A common modeling approach is to represent the problem as a graph, where the objective is to devise a collection of node-disjoint routes, one per vehicle, that both originate and terminate at the depot. In some variants, vehicles are subject to capacity constraints, leading to the Capacitated Vehicle Routing Problem (CVRP), which is probably the most extensively investigated version of the VRP.

A well-known generalization of the Vehicle Routing Problem (VRP) is the Multi-Depot Vehicle Routing Problem (MDVRP) (some recent works include Vieira, de Mello Bandeira, and da Silva Júnior (2021), Sadati, Çatay, and Aksen (2021), Arishi and Krishnan (2023), Dubey and Tanksale (2023), Zhang, Lin, and Li (2023), Baghbadorani and Haeri (2025), Liu et al. (2024b), Wei, Niu, Zhao, and Wang (2025) and Weng, She, Fan, Zhang, and Yun (2025)). In this variant, each depot is paired with its own fleet of vehicles, and the objective is to construct a collection of node-disjoint routes that covers all clients. Moreover, for each depot assigned k vehicles, exactly k routes must originate from it.

Another well-known variant of the VRP is the Vehicle Routing Problem with Pickup and Delivery (VRPPD), in which goods are transported from the depot to customers, but also from customers to the depot (some interesting works are Lehmann and Winkenbach (2024), Jiang, Dai, Yang, and Ma (2024), Zhang, Che, and Liang (2024), Boz, Calik, and Yusuf (2025), and Wang, Ding, Chen, Su, and Zhuang (2024)).

In the previously presented problem variants, we assume that all data referring to travel time, service time, customer demand, etc. are known with certainty before designing the routes. However, these assumptions are unjustified in many real-world situations, for instance, when considering the uncertainty associated with traffic conditions. In real-world applications, the VRP may involve one or more uncertain elements. Typically, these elements are represented as stochastic parameters in the problem formulation. When stochastic data are included in a problem, it is referred to as a Stochastic Vehicle Routing Problem (SVRP)

* Corresponding author.
E-mail addresses: bojeda@utec.edu.pe (B.H. Ojeda Rios), ecx@unicamp.br (E.C. Xavier).

https://doi.org/10.1016/j.eswa.2025.128258
Received 15 November 2024; Received in revised form 6 May 2025; Accepted 19 May 2025
Available online 31 May 2025
0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(Rios et al., 2021). Recent reviews of the literature on SVRP are provided by Gendreau, Jabali, and Rei (2014), Oyola, Arntzen, and Woodruff (2016a), Oyola, Arntzen, and Woodruff (2016b), Ulmer, Goodson, Mattfeld, and Thomas (2020), Rios et al. (2021), Tan and Yeh (2021), Soeffker, Ulmer, and Mattfeld (2022), Raza, Sajid, and Singh (2022), Konstantakopoulos, Gayialis, and Kechagias (2022), Ma, Zhang, and Guo (2022), Akhmetbek (2023), Adamo, Gendreau, Ghiani, and Guerriero (2024), Aydi, Chtourou, and Frikha (2024), Mardešić, Erdelić, Carić, and Đurasević (2024) and Yernar and Turan (2025).

Our problem is motivated by the context of food delivery services, where several new challenges arise. According to a report presented in Research and Markets (2024) the online food delivery services market size has grown rapidly in recent years. It will grow from $143.05 billion in 2023 to $159.46 billion in 2024 at a compound annual growth rate (CAGR) of 11.5 %. The historical period's growth can be attributed to the rise in robust economic growth in emerging markets, heightened internet penetration, and increased accessibility of mobile devices. In this new scenario, customers place orders through food delivery services (e.g., UberEats, iFood, and Grubhub), selecting an item from one of the associated restaurants. The food delivery service relays orders to the assigned courier and the corresponding restaurant. A courier then must pickup items from a set of restaurants (pickup points) and deliver them to the appropriate customers (delivery points).

Inspired by food delivery services, we introduce a new problem called the *Stochastic Capacitated Multi-Depot Vehicle Routing Problem with Pickup and Delivery* (SCMVRPPD), which is a generalization of the VRP. This problem is similar to the VRP with stochastic clients introduced by Gendreau, Laporte, and Séguin (1996). In the SCMVRPPD, a fleet of limited-capacity vehicles, each located in a different depot, must carry out pickups and deliveries of items. Each pickup point is uniquely associated with a single delivery point. Each pair of pickup/delivery vertices is associated with a client who may or may not place an order. If an order is placed, then a route must visit the corresponding pickup point and subsequently the delivery point. We consider a scenario in which the locations of the pickup and delivery points are known when designing the set of routes. However, for each pair of pickup/delivery points, we do not know a priori whether the associated client will place an order. Therefore, we assume that the actual need to visit each pair of pickup and delivery points is stochastic. In Fig. 1, we show the routes of three vehicles, each belonging to a different depot.

We consider the SCMVRPPD to be a two-stage stochastic optimization problem. In the first stage, we design a set of m routes, each starting and ending at a depot (*a priori* routes). In the second stage, the planned

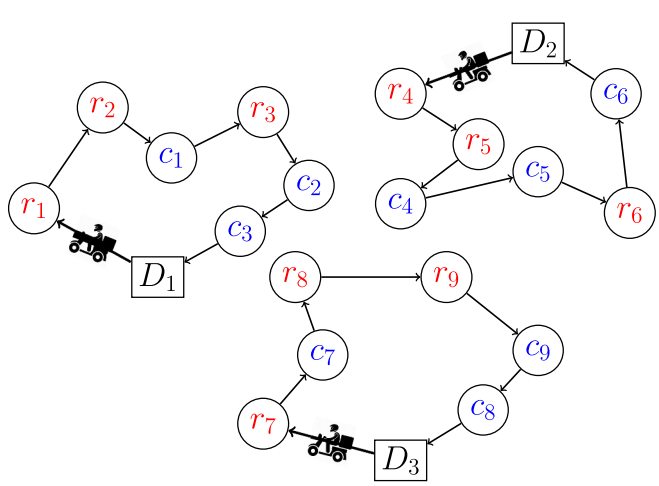

Fig. 1. Example of a feasible solution for the SCMVRPPD with three vehicles of the same capacity Q. Each vertex r_i represents a pickup point while c_i represents its corresponding delivery point.

routes are executed; however, uncertainty in pickup and delivery requests can lead to route failures. We say that a route fails if a pair of pickup and delivery points is unused (i.e., the associated client does not place an order). Another cause of route failure arises when a vehicle reaches its maximum capacity upon arriving at a pickup point. In the event of a failure, we apply different corrective strategies, which are presented in detail in Section 3.2.

The natural question is how to select the best set of *a priori* routes. In this sense, we look for a set of *a priori* routes of minimum total expected cost, that is, the total expected cost of the set of fixed routes plus the expected value of the extra cost that must be required for the execution of a particular correcting strategy.

The contributions of this paper can be summarized as follows. We present a new problem that models real-world situations in the food delivery service. Also, we propose three strategies to handle and update failed routes. We compare these strategies and select the one with the lowest computational cost. Based on this strategy, we propose a closed-form expression to compute the expected cost of an *a priori* route under a general probabilistic assumption. To address the problem, we propose three heuristic solutions: an Iterated Local Search (ILS), a Tabu Search (TS), and a Generalized Variable Neighborhood Search (GVNS). The first heuristic, ILS-VND, is an adaptation of the ILS heuristic presented by Subramanian, Drummond, Bentes, Ochi, and Farias (2010) for the VRP with simultaneous pickup and delivery. This heuristic approach uses the Variable Neighborhood Descent (VND) as a local search procedure. The second heuristic, TS, is based on the TABUSTOCH algorithm introduced by Gendreau et al. (1996) for the Vehicle Routing Problem with Stochastic Demands and Customers (VRPSDC). The third heuristic, GVNS, is based on the VNS heuristic presented by Carrabs, Cordeau, and Laporte (2007) for the Pickup and Delivery Traveling Salesman Problem with LIFO Loading. To our knowledge, this is the first article solving the SCMVRPPD.

The remainder of this paper is organized as follows. In Section 2, we present problems related to the one under consideration in this paper. In Section 3, we formally introduce the SCMVRPPD. In the same section, we present three failure correction strategies and propose an efficient formula to compute the expected cost of an *a priori* route. In Section 4, we describe the ILS and VNS algorithms in detail. Next, in Section 5, we present the TS algorithm. In Section 6, we report the results of our computational experiments. Finally, Section 9 concludes the paper.

2. Literature review

The closest related problem to the SCMVRPPD is the Multi-Vehicles and Multi-Depot Pickup and Delivery Problem with Time Windows (m-MDPDPTW), proposed by Dridi, Alaïa, Borne, and Bouchriha (2019). In the m-MDPDPTW, a fleet of geographically dispersed vehicles must fulfill requests from a set of nodes. There are two groups of nodes: supplier nodes and customer nodes. Request demands can be positive if they come from pickup nodes or negative if they come from delivery nodes. Each vehicle must start and end its tour at the same depot. In addition, pickup/delivery routes must respect constraints such as precedence (a pickup must always occur before its associated delivery), time windows, capacity, and so on. The goal is to minimize the total travel distance. To solve the problem, the authors propose an approach based on the Particle Swarm Optimization (PSO) method. Recently, other authors have published works similar to Dridi's, for example, Azarkish and Aghaeipour (2022) and Lv, Liu, and Li (2025).

In the Steiner Multi Cycle Problem (SMCP), introduced by Pereira, San Felice, Hokama, and Xavier (2018), we are given a complete graph and a collection of disjoint terminal sets. The objective is to find a set of disjoint cycles such that all vertices of the same terminal set lie on the same cycle. A particular case of the SMCP is the Restricted Steiner Multi Cycle Problem (R-SMCP), where each terminal set contains exactly two vertices, called pickup and delivery. To address the R-SMCP, the authors present heuristics based on geometric properties of the problem and

propose a GRASP algorithm. Furthermore, they provide a 4-approximation algorithm for the general SMCP. Recent works on this problem were presented by Lintzmayer, Miyazawa, Moura, and Xavier (2020), de Novaes Guimarães, San Felice, Hokama, and Xavier (2024), and Fernandes, Lintzmayer, and Moura (2024).

Another related problem is the Hamiltonian p-Median Problem (HPMP), proposed by Branco and Coelho (1990). The goal is to find p Hamiltonian cycles of minimum total cost such that each vertex belongs to exactly one cycle. The authors propose a mathematical formulation and present heuristics for tackling the problem. Subsequently, several works have considered this problem–for instance, Barbato, Canas, Gouveia, and Pesneau (2023), Barbato and Gouveia (2024), and He, Hao, and Wu (2024).

Many similarities also exist with variants of the Traveling Salesman Problem (TSP), such as the Capacitated Traveling Salesman Problem with Pickups and Deliveries (CTSPPD) (Anily & Bramel, 1999). The CTSPPD consists of n pickup points (blue points), n delivery points (red points), and a vehicle of limited capacity. Any solution must start and end at a designated blue point (the depot). Each blue point has one unit of load that must be delivered to a red point. The objective is to find a minimum-cost tour that does not violate capacity constraints. The CTSPPD is a particular case of the SCMDVRPPD when only one vehicle is available and the pickup and delivery points are deterministic. Another close variant of the TSP is the Multiple Traveling Salesman Problem (mTSP), which allows more than one salesman. A comprehensive review of the mTSP can be found in Bektas (2006), where the author describes several variants, examines how mTSP instances can be transformed into TSP instances, and discusses practical applications as well as heuristic and exact solution methods. The mTSP variant closest to the SCMVRPPD is the Multi-depot mTSP (MmTSP), see Lou, Xu, Yan, and Xiao (2021), Ramanathan, Suresh, and Rao (2023), Perez-Carabaza, Galvez, and Iglesias (2024), and Bajaj and Dhodiya (2025). The MmTSP generalizes the mTSP by allowing multiple depots. Moreover, each vehicle must serve a number of customers within a predefined interval.

Other related problems include the One-Commodity Pickup and Delivery Traveling Salesman Problem (1-PDTSP) (see Castro, Cire, and Beck (2020), Hernández-Pérez, Landete, and Rodríguez-Martín (2021), Hernández-Pérez and Salazar-González (2022), Palacio and Rivera (2022), Ghorpade and Corlu (2023), Domínguez-Martín, Hernández-Pérez, Riera-Ledesma, and Rodríguez-Martín (2024), and Domínguez-Martín et al. (2024)), the Traveling Salesman Problem with Pickup and Delivery (see Hernández-Pérez and Salazar-González (2004) and Li et al. (2024)), and the Dial-A-Ride Problem (see Xu et al. (2020), Rist and Forbes (2021), Schenekemberg, Chaves, Coelho, Guimarães, and Avelino (2022), Su, Dupin, and Puchinger (2023), Jodeau, Absi, Chevrier, and Feillet (2024), Liu, Quilliot, Toussaint, and Feillet (2024a), Bongiovanni, Geroliminis, and Kaspi (2024), Johnsen, Meisel, and Ehmke (2025), Grimm (2025a), Grimm (2025b)).

On the other hand, to the best of our knowledge, there is no existing literature on the SCMVRPPD. The closest problem is the Vehicle Routing Problem with Stochastic Demand, introduced by Bertsimas (1992).

This paper builds on the work of Rios, Xavier, Miyazawa, and Amorim (2020), who introduced the Stochastic Multi-Depot Vehicle Routing Problem with Pickup and Delivery (S-MDVRPPD) for the first time. In the S-MDVRPPD, vehicles have unlimited capacity, making it an unrealistic version of the VRPPD; however, this characteristic simplifies the calculation of the expected cost of an *a priori* route. In contrast, our work addresses a more realistic and challenging scenario by imposing capacity constraints on vehicles. This new feature requires devising cargo release strategies and designing efficient algorithms to handle the problem. In short, the fundamental difference between the S-MDVRPPD and the SCMDVRPPD lies in vehicle capacity constraints.

Below, in Table 1, we present a comparison of the main variants of vehicle routing problems with pickup and delivery (VRP with pickup and delivery) and some of their extensions. For each problem, the table includes the algorithm or approach used, as well as an analysis of its strengths and weaknesses, providing an overview of the different methods and their reported performance in the literature. Thus, it serves as a comparative reference for situating the newly proposed SCMDVRPPD within the broader spectrum of VRP and TSP variants discussed above.

2.1. Development and evolution of genetic algorithms in VRPs

Among the techniques presented in Table 1, genetic algorithms have undergone significant advancements in recent years, driven by the growing interest of the research community in proposing more efficient approaches. Notable contributions include the combination of genetic algorithms with graph theory, graph search techniques, clustering methods, and hybridization strategies with other metaheuristics, which has enabled more effective handling of various VRP variants and improved the quality of the solutions obtained.

2.1.1. Genetic algorithms Combined with clustering techniques

When scaling to hundreds or thousands of customers, searching the entire solution space becomes infeasible. To address this challenge, clustering techniques divide the global problem into smaller and more manageable subproblems by grouping customers. At the same time, genetic algorithms optimize the routes within these clusters by exploring solutions in an evolutionary manner. The combination of genetic algorithms with clustering is based on the divide-and-conquer principle, reducing dimensionality and improving computational efficiency.

The integration process between clustering techniques and genetic algorithms to solve the VRP is typically structured into three main phases, as outlined in Vidal, Crainic, Gendreau, and Prins (2014). The first phase, corresponding to the preprocessing through clustering, employs various methods to group customers geographically and reduce the complexity of the problem. These techniques include K-Means and its capacity-constrained variants (capacitated K-Means) (Ahmed, Seraj, & Islam, 2020), density-based clustering (DBSCAN) for identifying dense groups and removing outliers (Ester, Kriegel, Sander, Xu et al., 1996), agglomerative hierarchical clustering for adjusting the clustering granularity (Murtagh & Contreras, 2012), and constrained clustering, which ensures compliance with conditions such as vehicle capacity or time windows (Dinler & Tural, 2016).

In the second phase, optimization using genetic algorithms is applied within each cluster (Berger, Salois, & Begin, 1998). At this stage, solutions are encoded as permutations of the nodes belonging to the cluster, and crossover and mutation operators specifically adapted to the problem are employed, such as PMX (Partially Mapped Crossover), OX (Order Crossover), or swap-based exchanges, see Ombuki, Ross, and Hanshar (2006) and Goldberg and Lingle (2014). The fitness function evaluates criteria such as the total distance traveled, service time, or penalties for constraint violations.

Finally, in an optional third phase of global consolidation, cross-cluster node insertion operations can be performed to balance the load among routes. Additionally, further global heuristics–such as Large Neighborhood Search–can be applied to refine the solutions obtained after the execution of the genetic algorithm.

2.1.2. Genetic algorithms combined with graph theory

Crossover and mutation operators in genetic algorithms have been redefined to leverage structures inherent to graph theory, enabling more efficient route manipulation in problems such as the VRP. A prominent example is the Edge Assembly Crossover (EAX) operator, proposed by Nagata and Kobayashi (2013), which constructs a union graph from two parent solutions, from which alternating cycles are extracted and subsequently used to reconstruct feasible offspring through minimal connections between subgraphs. Another relevant operator is the Greedy Subtour Crossover (GSX) (Gutin & Punnen, 2006), which preserves common edges between both parents, facilitating a smooth transition between routes and prioritizing those with shorter distances, thanks to the direct

Table 1
Comparison of challenging VRP variants and their solution methods.

Problem name	Algorithm used	Strengths	Weaknesses	References
VRP with Pickup and Delivery	Genetic Algorithm	(a) Good capacity for combining multiple pickup and delivery constraints. (b) Flexible when dealing with changing data. (c) Easy integration of crossover and mutation operators.	(a) Complex parameter tuning (crossover, mutation, etc.). (b) Risk of stagnation in local optima without diversity strategies.	Park, Son, Koo, and Jeong (2021); Ibrahim, Putri, Farista, and Utama (2021); Ousmane, Moustapha, and Adama (2021); Dayarian, Savelsbergh, and Woensel (2020); Hemmelmayr, Cordeau, and Crainic (2022)
Multi-Depot Pickup and Delivery Problem with Time Windows	Ant Colony Optimization (ACO)	(a) Positive feedback (pheromone) guides the search effectively. (b) Adapts well to multiple depots and time windows. (c) Suitable for medium-sized problems	(a) Performance strongly depends on parameter tuning (pheromone evaporation). (b) Possible stagnation in local optima without proper diversification.	Li, Zhang, and Zhang (2020); Phuc and Thao (2021); Li, Bai, and Ouelhadj (2022); Wu and Gao (2023); Indravattana and Silawan (2023); Xue (2023);
Steiner Multi Cycle Problem	Mixed Integer Programming (MIP)	(a) Ensures optimality for small/medium instances.	(a) Exponential growth in runtime for large instances. (b) Requires careful modeling of constraints in MIP.	Accorsi, da Silva, and Mateus (2021); Fernandes et al. (2024); de Novaes Guimarães et al. (2024)
Hamiltonian p-Median Problem	Variable Neighborhood Search (VNS)	(a) Explores diverse neighborhoods, avoiding local minima. (b) Effective methodology combining intensification and diversification. (c) Suitable for location and routing combined perspective.	(a) Definition of neighborhoods and shaking steps can be complex. (b) High computational cost without well-defined stopping criteria.	Sahoo, Cordeau, and Laporte (2020); Ibrahim and Jarboui (2022);
Capacitated Vehicle Routing Problem with Pickup and Delivery	Tabu Search	(a) Handles simultaneous capacity and pickup/delivery constraints. (b) Memory mechanism prevents repetitive cycles. (c) Adaptive penalty adjustments.	(a) Highly sensitive to penalty function design. (b) Scalability limits for large problems unless acceleration methods are used.	Ge, Liu, and Liang (2020); Kyriakakis, Sevastopoulos, Marinaki, and Marinakis (2022); Ng, Kerk, Lim, and Tan (2024); Cornejo-Acosta, García-Díaz, Pérez-Sansalvador, and Segura (2023); Xu, Xiang, Li, and Li (2022)
Multiple TSP	Particle Swarm Optimization (PSO)	(a) Relatively fast convergence for mid-sized instances. (b) Manages multiple "travelers" simultaneously. (c) Easily parallelizable.	(a) May get trapped in local optima without perturbation. (b) Requires careful tuning of inertia and acceleration coefficients.	Yi, Gao, and Wang (2022), Qasimi (2024)
One-Commodity Pickup and Delivery Traveling Salesman Problem	Greedy randomized adaptive search procedure (GRASP) + VNS	(a) Constructs solutions (GRASP) combined with systematic neighborhood search (VNS). (b) Good balance between solution quality and computation time. (c) Addresses a single commodity with multiple routing constraints.	(a) Requires careful design of construction phase and neighborhood changes. (b) Potential computational overhead for large-scale instances.	Dang and Moukrim (2020); Ropke and Santini (2022)

incorporation of graph-based information, such as edge weights. In both cases, the main advantage lies in preserving the adjacency structure between nodes–i.e., topological connectivity–beyond the simple relative position in the solution encoding, thereby improving the quality and feasibility of the generated offspring.

2.1.3. Genetic algorithms combined with graph-based local search

Genetic algorithms are often combined with local search techniques to refine solutions and accelerate convergence, resulting in hybrid algorithms that are widely used in the Vehicle Routing Problem (VRP) (Vidal, Laporte, & Matl, 2020). Classical operators such as 2-opt, 3-opt, and Or-opt improve routes by reconnecting edges, while swap and relocate operators allow the exchange and repositioning of nodes between routes (Vidal et al., 2014). These improvements, closely linked to graph theory, reconfigure subpaths of the underlying graph to optimize the solution. To reduce computational cost, restricted neighborhoods are used, limiting the search to edges with low weights or high heuristic promise.

2.1.4. Genetic algorithms in multi-objective VRP problems

Several approaches have been proposed to address variants of the Vehicle Routing Problem (VRP) from a multi-objective perspective, incorporating complex constraints inherent to real-world logistics environments. Zhao, Bian, and Mei (2024) proposed the AMoGA-GIS algorithm (Adaptive Multi-Objective Genetic Algorithm with Greedy Initialization Strategy) to solve the VRP in heterogeneous green cities with time windows (HGCVRPTW). This approach introduces a greedy initialization strategy along with adaptive mutation operators that enhance both the diversity and quality of the generated solutions. Experimental results show that AMoGA-GIS outperforms other state-of-the-art algorithms in terms of efficiency and solution quality.

In a similar line of work, Labidi, Azzouna, Hassine, and Gouider (2023) developed an improved genetic algorithm tailored to the VRP with environmental considerations. This method incorporates penalty mechanisms to efficiently handle various constraints and employs specialized genetic operators to preserve population diversity throughout the evolutionary process. Computational experiments report significant improvements in the sustainability of the generated routes, as well as in the overall computational efficiency of the algorithm.

For their part, Rajesh, Jain, and Kotecha (2022) proposed a multi-objective approach specifically applied to the Electric Vehicle Routing Problem (EVRP). This model simultaneously considers the minimization of total travel time and the cost associated with vehicle charging. To achieve this, real-world variables such as traffic at charging stations and dynamic electricity prices are integrated. The algorithm

combines evolutionary techniques, such as genetic algorithms, with particle swarm optimization, achieving optimal solutions in highly complex scenarios.

The success of these approaches largely depends on the effective handling of complex constraints, which is implemented through various strategies. Among them, penalty functions stand out, as they introduce additional costs in the evaluation of solutions that violate constraints such as vehicle capacity or time windows.

3. Problem definition

First, we present a deterministic and static version of the SCMVRPPD, which we refer to as the Capacitated Multi-Depot Vehicle Routing Problem with Pickup and Delivery (CMVRPPD). Then we formally introduce the SCMVRPPD. Finally, we present a closed-form expression for calculating the expected cost of an *a priori* route under general probabilistic assumptions.

3.1. CMVRPPD

We start by defining the notation used in the remainder of the article. Let $G = (V, E)$ be a complete undirected graph, where $V = \{v_1, \dots, v_n\}$ is the vertex set, and $E = \{(v_i, v_j) : v_i, v_j \in V, v_i \neq v_j\}$ is the edge set. The vertex set $D = \{v_1, \dots, v_m\}$ represents the depots, and the set $V' = \{v_{m+1}, \dots, v_n\}$ represents the pickup and delivery points.

Let H^+ be the set of pickup points, H^- be the set of delivery points, and let $w = |H^+| = |H^-|$ be the size of these two sets (which have the same size). We assume that the vertex indices are arranged such that $v_i \in H^+$ if $m + 1 \leq i \leq m + w$, and $v_i \in H^-$ if $m + w + 1 \leq i \leq n$.

Each pickup point v_i is associated with a single delivery point v_{i+w} for $m + 1 \leq i \leq m + w$. Note that $w = |H^+| = |H^-| = |V'|/2$. We define a *route* as a sequence of pickup and delivery points that a vehicle could follow, starting from its depot with zero load, making pickups and deliveries of items, and finishing at the depot. Each edge (v_i, v_j) is associated with a travel cost d_{ij}. There are m homogeneous vehicles with the same capacity Q. Each vehicle is located in its own depot (each depot has only one vehicle assigned to it), so the number of depots and vehicles is the same.

We assume that every delivery point has a demand of one unit. The objective of the problem is to find m routes of minimum total cost such that the following restrictions are satisfied: (a) each vehicle route begins and ends at its depot, (b) each pickup and delivery point is visited only once by exactly one vehicle, (c) each associated pickup and delivery pair, e.g. $\{v_i, v_{i+w}\}$ for $m + 1 \leq i \leq m + w$, must belong to the same route, (d) each pickup point precedes its associated delivery point on the route, and (e) for every vehicle route, the total load never exceeds the vehicle's capacity Q.

In a real situation, restaurants (pickup points) want to sell their products to as many customers (delivery points) as possible. For this reason, our instances replicate pickup and delivery points multiple times. This way, we replicate the same pickup point for every delivery point that orders from it; similarly, a customer can place an order in more than one restaurant.

This problem is NP-hard since it generalizes the traveling salesman problem (TSP) when there is a single depot, the vehicle's capacity is large enough, and the associated pickup and delivery points are in the same location.

3.2. SCMVRPPD

We define the particular SCMVRPPD considered in this paper. In this problem, each associated pair of pickup and delivery points $\{v_i, v_{i+w}\}$ has a probability of presence P_i, for $m + 1 \leq i \leq m + w$. Consequently, if the pickup point v_i is absent, then the delivery point v_{i+w} is also absent. The presence or absence of $\{v_i, v_{i+w}\}$ is revealed only when the vehicle is leaving the vertex preceding v_i on the route. We consider the SCMVRPPD as a two-stage stochastic problem. In the *first stage*, a set of (*a priori*) routes is computed, satisfying constraints (a) - (d) of the CMVRPPD (see Section 3.1). In the *second stage*, these first-stage routes are followed as planned, but due to the uncertainty of vertex occurrence, a route can fail. We say that a route fails if a pickup point and its corresponding delivery point do not appear, or if restriction (e) is violated (i.e., when the load at some point exceeds the vehicle's maximum capacity). To correct failed routes, we present route-update strategies. If a pair of pickup and delivery points does not appear in some *a priori* route, they are simply skipped and we continue to the next vertex. Fig. 2 shows the skip strategy for updating an *a priori* route.

In this problem, each time the vehicle reaches a pickup point, a unit of load is added. Similarly, each time the vehicle reaches a delivery point, a unit of load is removed. If the *a priori* route fails because the vehicle reaches maximum capacity when visiting a pickup point, we consider three cargo release strategies, as shown in Fig. 3. To explain these strategies, suppose a pickup-delivery pair v_p, v_{p+w} and let $d(v_p, v_{p+w})$ be the distance between them.

Strategy (a) – If the vehicle arrives at a pickup point and the vehicle's load is greater than or equal to Q, then a penalty equivalent to $2 \times d(v_p, v_{p+w})$ must be paid. The penalty corresponds to the cost paid to a third-party service to make the delivery from v_p to v_{p+w}. Employing third-party services for deliveries that a company cannot perform is common in transport logistics, since hiring a third party can be cheaper than using the company's own resources.

Strategy (b) – If the maximum load of the vehicle is reached in v_p, then the vehicle delivers all loaded items to their respective delivery points. Once the vehicle is completely empty, the route is resumed and

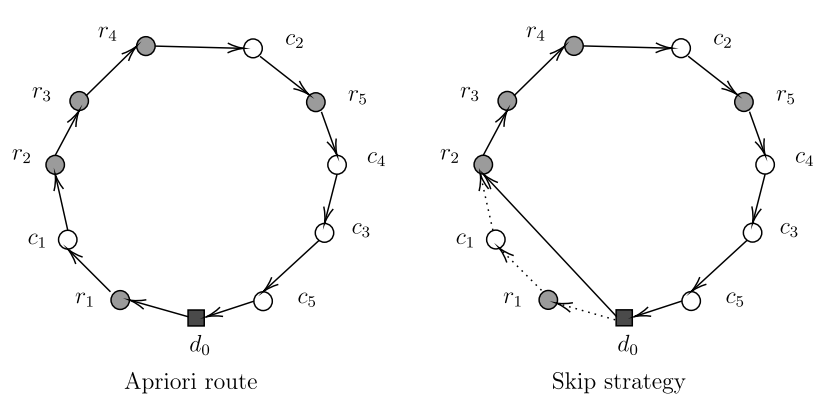

Fig. 2. The gray vertices are the pickup points (r_i) and the white vertices are delivery points (c_i). The figure on the left presents an example of an *a priori* route. When leaving the depot, it is revealed that the pair r_1, c_1 is not present in the route. The figure on the right shows the application of the skip strategy to vertices r_1 and c_1.

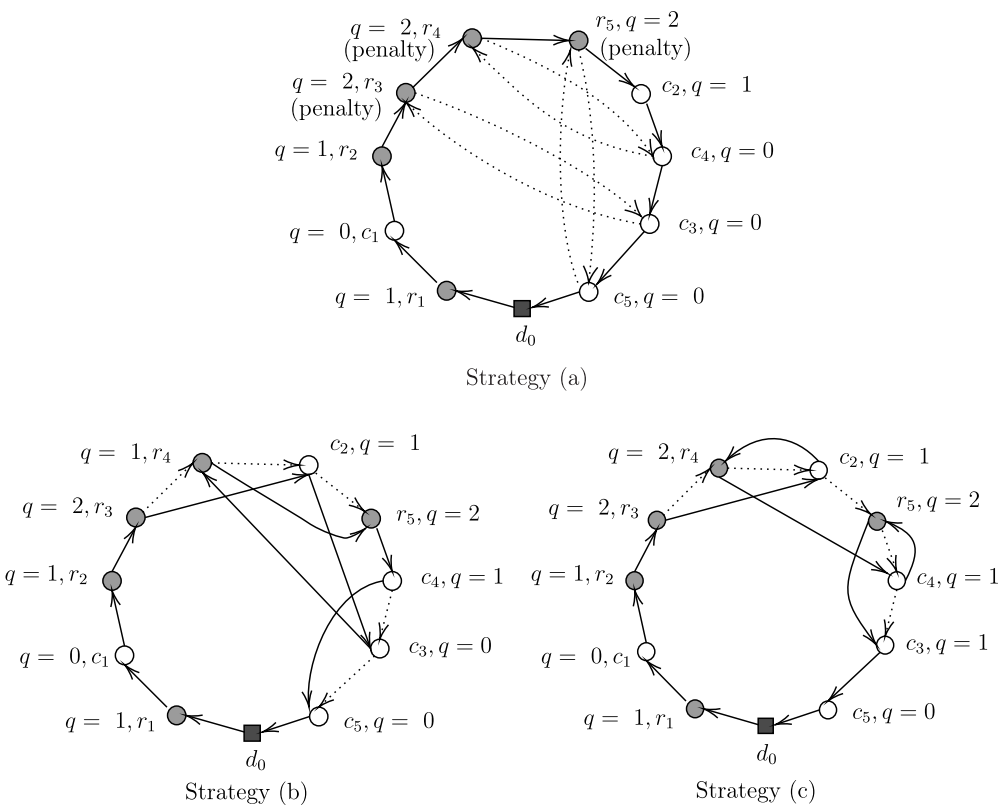

Fig. 3. In this example, we assume $Q = 2$. The gray vertices are pickup points (r_i), the white vertices are delivery points (c_i), d_0 is the depot, and q is the current load. In Strategy (a), the vehicle reaches maximum capacity at r_3, r_4, and r_5, then a penalty is paid at each of these nodes before the vehicle continues along the original route. In Strategy (b), the vehicle reaches maximum capacity at r_3 and r_5, empties completely, and continues the route as planned from the next unvisited vertex. Finally, in Strategy (c), the vehicle reaches maximum capacity at r_3, r_4, and r_5. At each of these vertices, the vehicle removes one unit of load, then continues from the next unvisited vertex.

followed as planned. Unlike strategy (a), the vehicle does not return to the point v_p.

Strategy (c) – If the maximum load of the vehicle is reached in v_p, then the vehicle makes a partial delivery to the closest delivery vertex v_d on the route for one of the current units. The vehicle then proceeds to the next unvisited vertex on the route. In this strategy, the vehicle is partially unloaded each time capacity is reached.

We compared the three strategies presented above and selected strategy (a) to be used in later algorithms, because it takes much less computational time to determine the cost of an *a priori* route (see Appendix A). The objective of the problem is to find a first-stage solution that minimizes the expected value of the second-stage solution.

3.3. The expected cost of an a priori route using strategy (a)

In this section, we show how to compute the expected cost of an *a priori* route under strategy (a) efficiently. The result is similar to the one presented in Rios et al. (2020), but we extend it to include penalty costs when the vehicle becomes full at a pickup point, since in the previous work, vehicle capacities were not considered. For completeness, we restate the main notation and the main result here.

Given an *a priori* route $s = (v_0, v_1, \dots, v_{2q}, v_0)$, where v_0 is a depot, let l_s be the cost/length of s. Our goal is to compute efficiently the expected length $E[l_s]$ of route s, given that each pair of pickup and delivery points in this route has a probability of occurring during its execution.

We assume that R is the set of pickup points that appear in s ($|R| = q$). The pickup points are labeled in superscript according to their order in s, from r^1 to r^q. Likewise, C is the set of corresponding delivery points, labeled from c^1 to c^q in the order they appear in s. We also use the following notation: if v_i is a pickup point, we write $r(v_i)$, and its corresponding delivery point is denoted by v_i^-. If v_i is a delivery node, we write $c(v_i)$, and its corresponding pickup point is denoted by v_i^+. Finally, given a subsequence $s_i^j = (v_i, v_{i+1}, \dots, v_j)$ of s, let $R(s_i^j)$ be the set containing the pickup points that appear in s_i^j, as well as the pickup points of the delivery vertices that appear in s_i^j. Notice that $|R(s_i^j)| \le j - i + 1$, and it is strictly smaller only if a pickup point appears in s_i^j and its delivery point also appears.

Note that we have the following relationship for a pair of pickup and delivery points r and c: $P(r) = P(c)$, $P(c|r \text{ appears}) = P(r|c \text{ appears}) = 1$, and $P(c|r \text{ not appear}) = P(r|c \text{ not appear}) = 0$.

The penalty that must be paid when a vehicle reaches maximum capacity at a pickup point r is denoted by $\phi(r) = 2 \times d_{r,c}$, where c is the delivery point associated with r. The probability that the vehicle becomes full at vertex r is denoted by $\gamma(r)$. Notice that when computing $\gamma(r)$ for some vertex r, we can disregard all pairs of pickup/delivery vertices where both appeared before r in the route. For example, suppose the route $(v_0, r^1, r^2, r^3, c^2, c^3, r^4, \dots)$ and we want to compute $\gamma(r^4)$. We can safely disregard the pairs r^2/c^2 and r^3/c^3 because their deliveries are done before reaching r^4, but we cannot disregard node r^1.

Finally, the probability of traversing an arc (v_i, v_j) for any two vertices in s is denoted by $P(v_i, v_j)$. We can compute $E[l_s]$ as follows.

Theorem 1. *Given an a priori route $s = (v_0, v_1, \dots, v_{2q}, v_0)$, then:*

$$E[l_s] = \sum_{i=1}^{q} d_{v_0,r^i} P(r^i) \prod_{k=1}^{i-1} (1 - P(r^k)) + \sum_{i=1}^{q} d_{c^i,v_0} P(c^i) \prod_{k=i+1}^{q} (1 - P(r^k))$$

$$+\sum_{i=1}^{2q}\sum_{j=i+1}^{2q} d_{v_i,v_j}\, P(v_i,v_j) \tag{1}$$
$$+\sum_{i=1}^{q}\gamma(r^i)\,\phi(r^i)$$

where:

$$P(v_i,v_j)=\begin{cases}0, & \textit{(a) or (b)}\\ P(v_i)\times\prod_{v\in R(s_{i+1}^{j-1})}(1-P(v)), & \textit{(c)}\\ P(v_i)P(v_j)\times\prod_{v\in R(s_{i+1}^{j-1})}(1-P(v)), & \textit{(d)}\end{cases} \tag{2}$$

$$\gamma(r^i)=\begin{cases}0, & i=0,\dots,Q-1\\ p_i\sum_{k=Q}^{i} f(i-1,k-1), & i\geq Q,\end{cases} \tag{3}$$

and $f(i-1,k-1)$ is defined as $Pr\{$exactly$i-1$ pickup points among the pickups $1,\dots,k-1$have nonzero demand$\}$. As explained above, only pickup points whose associated deliveries appear after k on the route are considered. The general value of $f(m,w)$ is defined recursively, for $m=1,\dots,n$ and $w=1,\dots,m$, as follows:

$$f(m,w)=p_w\, f(m-1,w-1)+(1-p_w)\, f(m,w-1) \tag{4}$$

with the initial conditions

$$f(m,m)=\prod_{i=1}^{m}p_i,\quad f(0,m)=\prod_{i=1}^{m}(1-p_i). \tag{5}$$

Proof. In Eq. (1), we are basically computing the probability that each edge between vertices in s will be traversed, and multiplying this probability by the edge's cost. We have four terms in this equation:

In the first term, we compute the expected cost of each possible initial edge of the route, such that if the route starts with (v_0,r^i), then all previous pickup points r^k, $k=1,\dots,i-1$ must not appear. In the second term, we compute the expected cost of each possible final edge in the route, similarly considering future pickups. In the third term, we compute the expected cost of each edge for any pair of vertices in the route. Fig. 4 represents all possible cases for vertices v_i and v_j. Since cases (a) or (b) do not occur in practice, the expected cost of (v_i,v_j) in those cases is zero. In case (c), the probability of going directly from $r(v_i)$ to $v_j=v_i^-$ is $P(v_i)$ times the probability of no pickups between v_i and v_j. In case (d), if v_i and v_j are unrelated, the probability of edge (v_i,v_j) is $P(v_i)P(v_j)$ times the probability of no intermediate pickups occurring. Finally, in the last term, $\gamma(r^i)$ represents the probability that the load reaches Q at r^i.

For the calculation of $\gamma(r^i)$, we only consider the pickup points whose associated deliveries appear after r^i in the route (see the example in Fig. 5). In this procedure, we remove from s the pickup and delivery pairs that appear entirely before r^i, obtaining a new route s'. In s', the position of r^i might change to r^j, so $\gamma(r^i)=\gamma(r^j)$. Clearly, $\gamma(r^j)=0$ for $j=0,\dots,Q-1$. For $j\geq Q$, if exactly $k-1$ pickups among the previous $j-1$ occur, the vehicle becomes full at r^j. This leads to Eq. (3). The probabilities $f(m,r)$ are computed recursively with Eq. (4) and the initial conditions in (5). □

4. ILS and VNS Algorithms

In this section, we introduce the local search operators, perturbation operators, and heuristics used by GVNS and ILS-VND. Finally, we present both algorithms proposed to deal with the SCMVRPPD.

4.1. Initial solution generation

The method used to generate an initial feasible solution is based on the work of Kuo and Wang (2012). For the construction of the initial solution, three steps are followed. The first step is the node assignment, where the probabilities of assigning the vertices to the depots are calculated. The second step deals with the grouping of the nodes around each depot. In this step, the probabilities computed in the first step are used to assign nodes to depots. The last step is the node sequencing: here, the grouped nodes are ordered to generate cycles. Each of these steps is described in more detail in the following subsections.

4.2. Nodes assignment

We adapted the method proposed by Kuo and Wang (2012) for assigning clients to a depot based on probabilities. The idea behind this method is to make the assignment of nodes to depots more flexible. To calculate the probability of assigning a node to a depot, we define

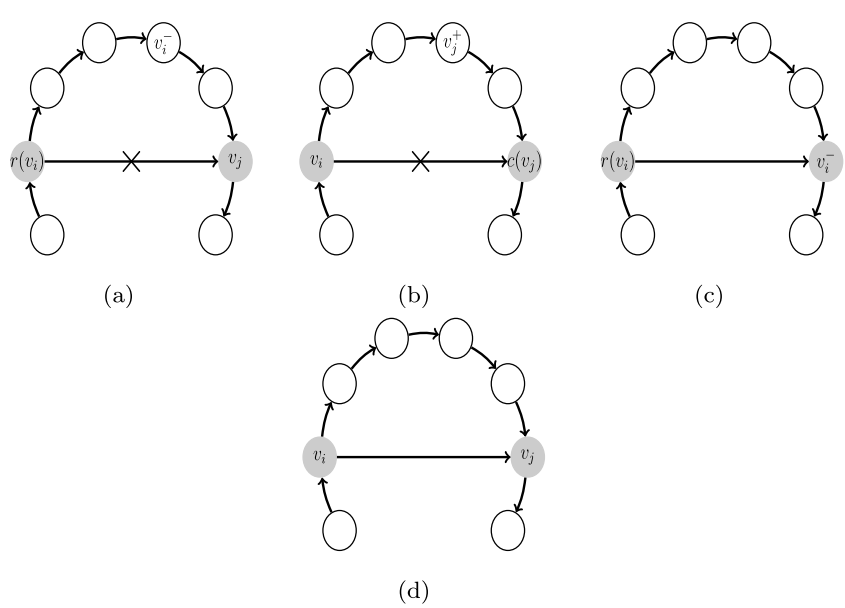

Fig. 4. In (a) v_i is a pickup node and v_j appears after v_i's corresponding delivery node v_i^-. In (b) v_i is any vertex, v_j is a delivery node and v_j^+ appears after v_i. Both situations, (a) and (b) do not occur, since in (a), if $r(v_i)$ is present in the route then v_i^- must appear as well, and in (b) if $c(v_j)$ is present then v_j^+ must be present as well, so going from v_i directly to v_j skipping vertices in between is not a valid route. In (c) we have the case where v_i is a pickup point and v_j is its corresponding delivery point. In (d) we have all other cases that do not belong to one of the previous cases.

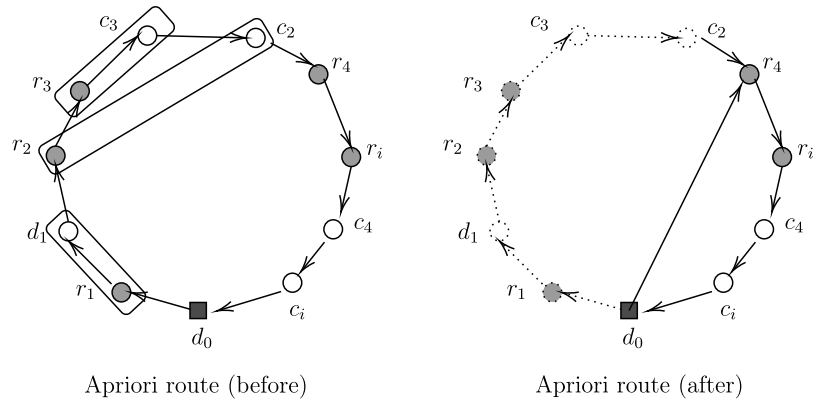

Fig. 5. The gray vertices are the pickup points (r_i), the white vertices are delivery points (c_i), and d_0 is the depot. The example shows the elimination of nodes $r_1, d_1, \ldots, c_2$ before computing $\gamma(r_i)$. The figure on the left shows an *a priori* route before removing the associated pickup and delivery points that precede vertex r_i. In this figure, we indicate the pickup and delivery pairs to be removed ((r_1, c_1), (r_2, c_2) and (r_3, c_3)). The figure on the right shows the *a priori* route used to calculate $\gamma(r_i)$. Only the pickup points appearing before r_i on the route ($\{r_4\}$) are used to calculate $\gamma(r_i)$.

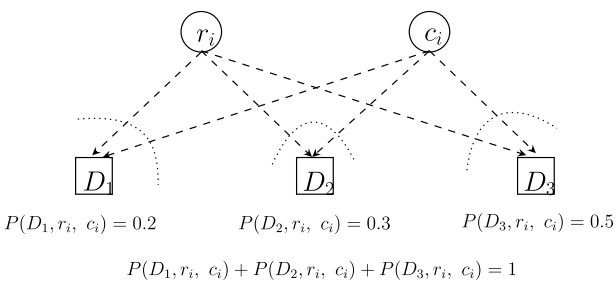

Fig. 6. Example of the node assignment method.

the following functions. Let D_a be a depot, (r_i, c_i) a pair of associated pickup and delivery points, and $d(x, y)$ the distance between two points. Let $d(D_a, r_i, c_i) = d(D_a, r_i) + d(D_a, c_i)$ be the sum of the distances from r_i and c_i to the depot D_a. Let $\overline{d(D, r_i, c_i)} = \sum_{j=1}^{|D|} d(D_j, r_i, c_i) / |D|$ be the mean of the distances from (r_i, c_i) to all depots. The probability that a pair (r_i, c_i) is assigned to the depot D_a is defined by the following equation:

$$P(D_a, r_i, c_i) = \frac{\max\left\{\overline{d(D, r_i, c_i)} - d(D_a, r_i, c_i), 0\right\}}{\sum_{a=1}^{|D|} \max\left\{\overline{d(D, r_i, c_i)} - d(D_a, r_i, c_i), 0\right\}} \tag{6}$$

Fig. 6 illustrates an example of the probability calculation for assigning a pair (r_i, c_i) to three depots. After applying Eq. (6), the sum of the assignment probabilities equals 1. Depending on the instance, the denominator of Eq. (6) can be zero. An example of this type of instance occurs when several depots are geographically located in the same position. To deal with such cases, we set the assignment probability for each depot equal to $1/|D|$.

Each pair of pickup and delivery (r_i, c_i) is assigned to a depot with the probabilities computed above.

4.3. Nodes sequencing

Once all nodes are assigned to depots, the next step is to create a route for each depot and its assigned nodes. The procedure for sequencing the nodes is inspired by the nearest neighbor algorithm. The initial route consists of a single node: the depot. The idea is to select the closest node to the last node of the sequence, add the selected node at the end of the sequence (without breaking any constraints of the problem), and then mark it as visited. The above steps are repeated until all vertices have been visited.

4.4. Local search

We propose four neighborhood structures $\{N^{(1)}, \ldots, N^{(4)}\}$ that allow movements from one feasible solution to another. These operators are exhaustively executed and are classified based on the type of movement they allow. We classify the operators as *inter-route*, *intra-route*, or *inter&intra-route*. The inter-route operators $\{N^{(1)}, N^{(2)}\}$ allow movements of nodes between different cycles. In contrast, the intra-route operator $\{N^{(3)}\}$ allows movements of nodes in the same cycle. The inter&intra-route operator $\{N^{(4)}\}$ combines the two previous types of movements, meaning a node can move to a different position in the same route or move to a different route. A move allowed by the $N^{(4)}$ operator involves removing a node from its route and inserting it before or after one of its p nearest neighbors.

In all neighborhood operators, non-feasible solutions can be generated. In our implementation, we simply discard such solutions.

The neighborhood operators considered are:

Shift(1,0) – N^1 – This operator removes a pair (i^+, i^-) from one route and then inserts them into a different route. Fig. 7 presents an example of this operator, where the associated pair (i^+, i^-) is moved from route A to route B. Route A begins at depot D_1, and route B begins at depot D_2. We call this process "relocation." Specifically, k nearest neighbors of i^+ are selected; we denote one of these neighbors by i^+_{ref}. For each of the k neighbors, i^+ is inserted after i^+_{ref}. On the other hand, i^-_{ref} is a vertex between i^+ and the depot. We insert i^- immediately after each i^-_{ref}. The best move of (i^+, i^-) is selected. Moreover, for each removed pair (i^+, i^-) from one route, we try to insert them into every other route, and the final result is the one with the best cost.

Swap(1,1) – N^2 – This operator exchanges two pairs of pickup and delivery from different routes. For example, suppose that the pair (i^+, i^-) belongs to route A and the pair (j^+, j^-) belongs to route B. The Swap(1,1) operator first swaps the pickup point i^+ with j^+. Then the operator inserts the delivery point i^- after i^+ in the best position on route B. Analogously, point j^- is inserted in the best position on route A. Fig. 8 illustrates an example of this operator, where the pair (i^+, i^-) is exchanged with the pair (j^+, j^-).

2-opt – N^3 – The idea behind the 2-opt operator is to reorder a route that crosses over itself. This operator can generate invalid paths; some of these can be corrected by inverting the involved edges and/or vertices. Fig. 9 presents an example of the 2-opt operator. This example shows a route where edges (r_1, r_3) and (c_1, c_2) intersect. To reorder the route, these edges are removed. Then two new edges, (r_1, c_1) and (r_3, c_2), are inserted in the route. Finally, to generate a valid route, the direction of edges (r_2, c_1) and (r_3, r_2) is reversed (see the blue dotted line). The

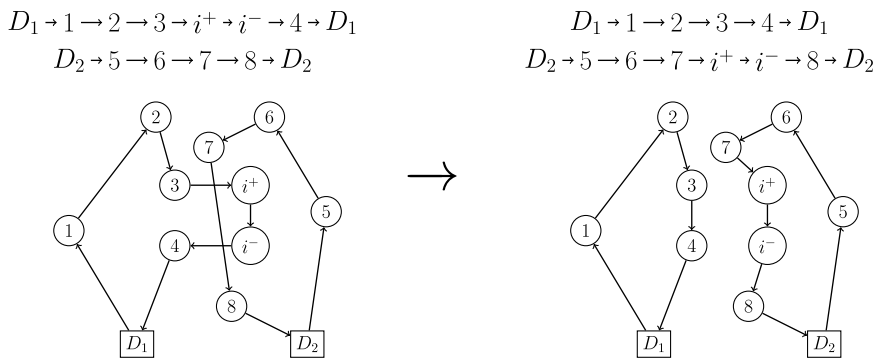

Fig. 7. An example of shift(1,0) operator.

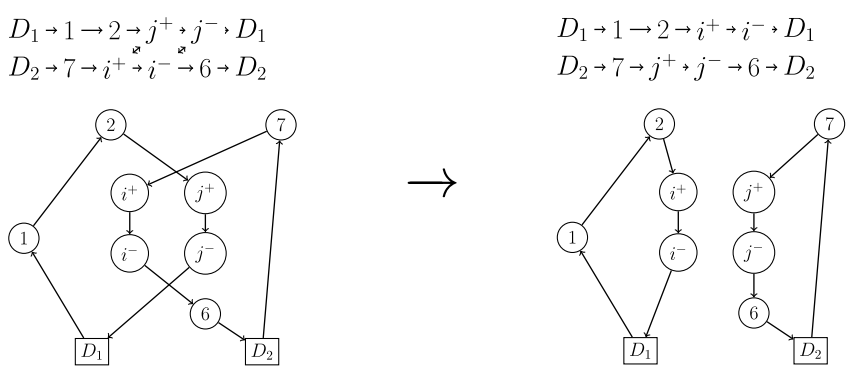

Fig. 8. An example of nodes exchange operator.

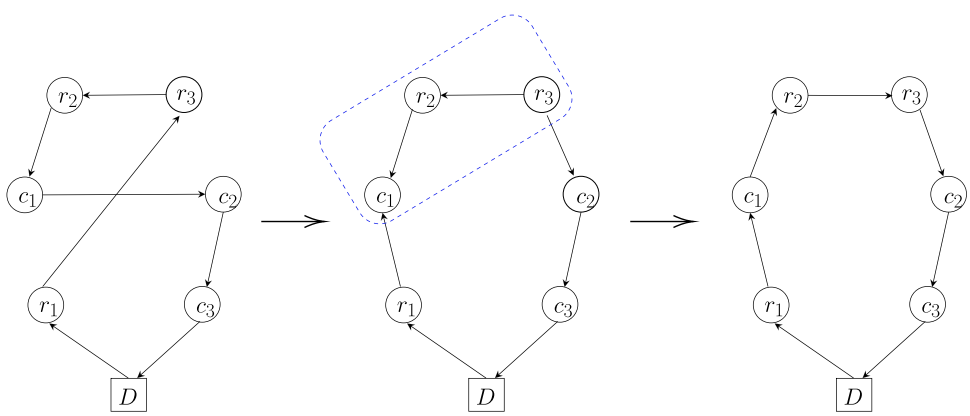

Fig. 9. Example of the 2-opt operator.

cross-removing effects are presented in Pan and Xia (2014). Invalid paths that cannot be corrected are discarded.

Mix-Shift(1,0) – N^4 – This operator is similar to Shift(1,0), except that here it is allowed to move a pair within its own route.

Given the versatility of the *Mix-Shift(1,0)* operator, and based on a neighborhood structure proposed by Gendreau et al. (1996), we present two heuristics: *Node-Insertion* and *Random Mix-Shift*.

The **Node-Insertion** Heuristic (Algorithm 1) changes the positions of a pair of associated pickup and delivery points (r, c) in a solution. Given the p nearest neighbors of r (which may belong to different routes), for each neighbor we insert r after it, and select as the best neighbor (denoted by r_{ref}) the one that yields the lowest solution cost. Then a vertex between r_{ref} and the depot on that route is randomly selected, denoted by c_{ref}. The point c is inserted immediately after c_{ref}. Note that after the execution of Node-Insertion, the vertices r and c belong to the same route, so the constraints of the problem are never violated.

In the **Random Mix-Shift** heuristic (Algorithm 2), we start with a current solution s and randomly select a pair (r, c) to use in the Node-Insertion heuristic. A new solution s' is generated and becomes the new current solution if it improves the existing one. Also, the best overall solution s^* is updated if s' is better than s^*. To explore a larger neighborhood, the heuristic may accept s' as the new current solution if its cost is at most α times the best overall solution s^*.

Our local search is based on the VND algorithm. Mladenović and Hansen (1997) proposed VND as a procedure that systematically modifies the neighborhood structure in a deterministic way. The proposed VND is presented in Algorithm 3.

Algorithm 1: Node-insertion.

```
Let (r, c) be a pair of pickup and delivery of a
  solution s and let P be the set of the closest p
  neighbors of r.;
r_ref = SelectRandomCloseNeighbor(r, p);
k = number of vertices between r_ref and the end depot;
Randomly select one of the k vertices between r_ref
  and the depot;
c_ref = SelectRandomNode(r_ref, k);
Insert r immediately after r_ref; insert c immediately
  after c_ref.;
Insert(r_ref, r);
Insert(c_ref, c);
```

Algorithm 2: Random mix-shift.

```
Let f(s) be the cost function of a solution s.;
for k := 1 ... MaxIterShift do
    r, c := SelectRandomPair(r, c);
    s' := Node-Insertion(r, c, s, p);
    if f(s') < f(s*) then
        s := s'; s* := s'; k := 1;
    else
        if f(s') < α f(s*) then
            s := s';
        else
            s := s*;
```

Algorithm 3: VND.

```
Let |N| be the number of neighborhood structures and
  s a current solution.;
Let f(s) be the cost of solution s.;
for N^k ∈ [N^1, N^2, N^4] do
    Find the best neighbor s' of s ∈ N^k.;
    if f(s') < f(s) then
        s := s';
        k := 1;
        N^3 is used as an intensification operator.;
        s' := 2-opt(s);
        if f(s') ≤ f(s) then
            s := s'
```

4.5. Perturbation mechanism

In the ILS-VND heuristic, we use two perturbation mechanisms. Every time the *perturb()* function is called, one of the following operators is randomly selected and applied.

Double-Swap – $P^{(1)}$ – Two Swap(1,1) operators are performed in sequence.

Reverse – $P^{(2)}$ – This operator inverts the direction of the edges of a randomly selected route. Fig. 10 presents an example of the reverse operator. Since reversing the edges generates an invalid route, each pickup point r_i is swapped with its associated delivery point c_i.

4.6. Approximations

In this section, we present approximations for computing the expected cost of the solution after applying the Shift(1,0), Mix-shift(1,0), and Swap operators. Note that computing the expected value of a solution using Eq. (1) is expensive. Moving a pickup and delivery pair not only affects the cost of their immediate neighbors, but also the cost of each node in the tour.

Suppose we want to insert the associated pickup and delivery pair (r, c) into a route (see Fig. 11). We propose low computational cost approximations that find values close to the real expected cost.

To simplify the definition of these approximations, we redefine the notation introduced in the previous section. We denote by P_v the probability of presence of vertex v in the solution.

The approximations for the operators *Shift(1,0)* and *Mix-Shift(1,0)* are based on Eqs. (7)–(9) (see Fig. 12).

$$f_1(x, a, b, c, d) = d_{a,b} + d_{c,x} + d_{x,d} - (d_{a,x} + d_{x,b} + d_{c,d}) \tag{7}$$

$$f_2(x, a, b, c, d) = d_{a,b}P_x + d_{c,x}P_x + d_{c,d}P_x - (d_{a,x}P_x + d_{x,b}P_x + d_{c,d}P_x) \tag{8}$$

$$\begin{aligned} f_3(x, a, b, c, d) = {} & d_{a,b}P_aP_b + d_{c,x}P_cP_x + d_{c,d}P_cP_d \\ & - (d_{a,x}P_aP_x + d_{x,b}P_xP_b + d_{c,d}P_cP_d) \end{aligned} \tag{9}$$

Assume that the pickup and delivery pair to be moved is x^+ and x^-. We denote by a and b the immediate neighbors of x^+ in its current (source) route, and by c and d its immediate neighbors in the target route (similar to Fig. 12(a)). Analogously, a' and b' are the immediate neighbors of x^- in the source route, and c' and d' are the neighbors in the target route. Since operators Shift(1,0) and Mix-shift(1,0) basically perform the same movement, we propose the following three approximations for both:

$$A_i = f_i(x^+, a, b, c, d) + f_i(x^-, a', b', c', d'), \quad 1 \le i \le 3 \tag{10}$$

To assess the quality of the approximations A_1, A_2, and A_3, we generated 312 randomly generated solutions, involving instances of 10 to 50 vertices, to find the best correlations with the true expected cost. We use the correlation to select the best approximations.

For both operators, approximation A_2 yields the best correlation with the true cost. Specifically, the correlation found for Shift(1,0) was 0.749, and for Mix-Shift it was 0.511.

Swap(1,1): Let x be a vertex in one route and y a vertex in another route, and suppose we want to swap their positions. We use the approximations given by Eqs. (11)–(13) as estimates of the real cost of this swap (see Fig. 13(a) and (b)).

$$g_1(a, x, b, c, y, d) = d_{a,y} + d_{y,b} + d_{c,x} + d_{x,d} - (d_{a,x} + d_{x,b} + d_{c,y} + d_{y,d}) \tag{11}$$

$$\begin{aligned} g_2(a, x, b, c, y, d) = {} & d_{a,y}P_xP_y + d_{y,b}P_xP_y + d_{c,x}P_xP_y + d_{x,d}P_xP_y \\ & - \left(d_{a,x}P_xP_y + d_{x,b}P_xP_y + d_{c,y}P_xP_y + d_{y,d}P_xP_y\right) \end{aligned} \tag{12}$$

$$\begin{aligned} g_3(a, x, b, c, y, d) = {} & d_{a,y}P_aP_y + d_{y,b}P_yP_b + d_{c,x}P_cP_x + d_{x,d}P_xP_d \\ & - \left(d_{a,x}P_aP_x + d_{x,b}P_xP_b + d_{c,y}P_cP_y + d_{y,d}P_yP_d\right) \end{aligned} \tag{13}$$

Eqs. (11)–(13) are used to compute approximate costs of swapping pairs of pickup and delivery (x^+, x^-) and (y^+, y^-). Fig. 13(c) and (d) illustrate this. We propose the approximations presented in (14). Using the same set of 312 instances to evaluate the approximation that best correlates with the real costs, Eq. (12) achieves the best correlation ($r = 0.924$).

$$B_i = g_i(a, x^+, b, c, y^+, d) + g_i(a', x^-, b', c', y^-, d'), \quad 1 \le i \le 3 \tag{14}$$

4.7. ILS-VND

Iterated Local Search (ILS) is a metaheuristic that stands out for its simplicity, flexibility, and robustness, significantly enhancing the performance of local search algorithms (Torres-Tapia, Montoya-Torres, & Ruiz-Meza, 2022). Its methodology is based on three main stages. First, a feasible initial solution is generated using either a constructive procedure or a random approach. Next, an intensive local search is applied to optimize this solution until a local optimum is reached. However, since

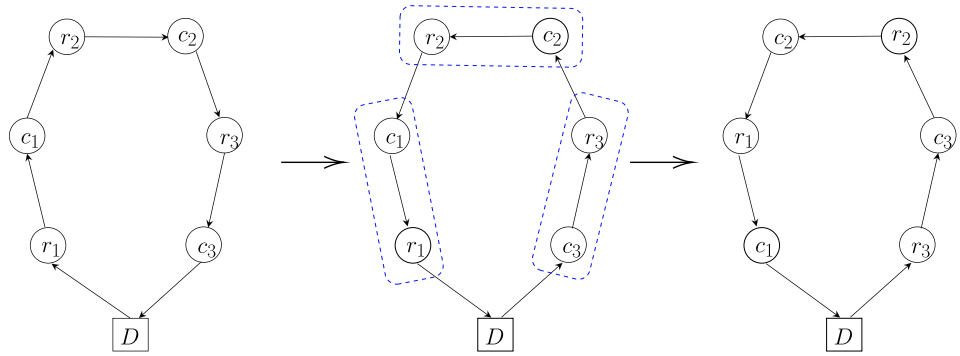

Fig. 10. Example of the reverse operator.

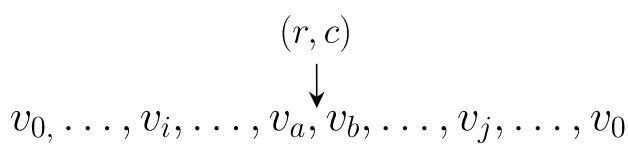

Fig. 11. Example of inserting a pair of pickup (r) and delivery (c) points in a route.

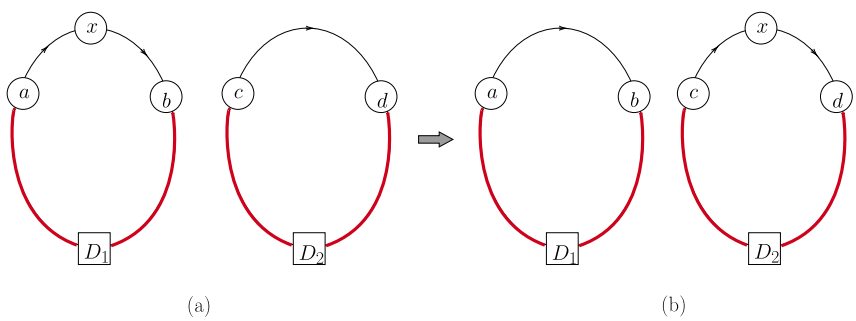

Fig. 12. Example of moving a node from one route to another. Figure (a) shows the position of node x before the move. Figure (b) shows the position of node x after the move. Red lines represent subtours and black arrows represent arcs.

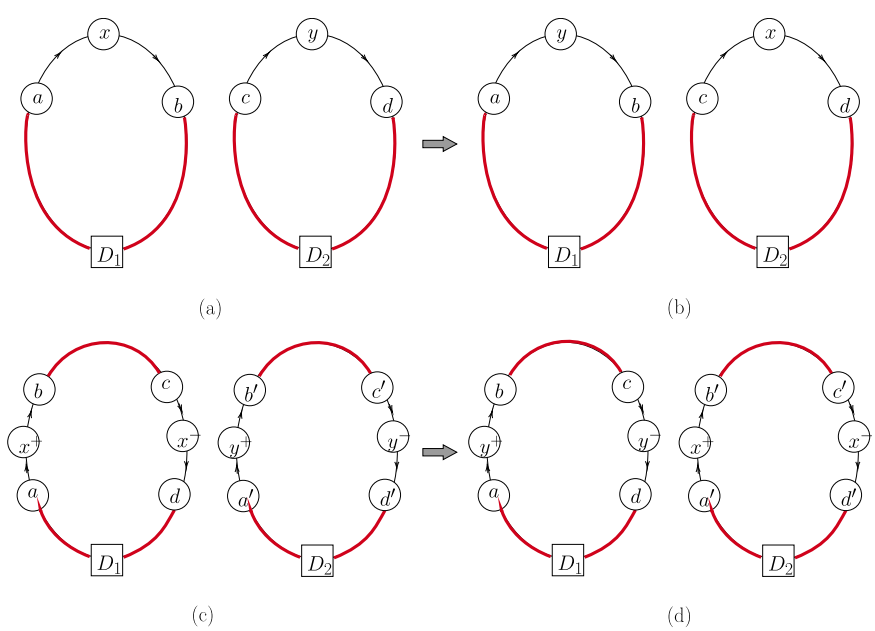

Fig. 13. Figure (a) shows the position of nodes x and y before the move. Figure (b) shows the position of x and y after the move. Figure (c) shows the pairs (x^+, x^-) and (y^+, y^-) before the swap. Figure (d) shows their positions after the swap. Red lines represent subtours and black arrows represent arcs.

local optima can deviate from the globally optimal solution, ILS incorporates a perturbation phase that controlledly modifies the current solution, allowing the algorithm to escape these local optima and explore new regions of the solution space. The continuous iteration between local search and perturbation effectively balances intensification and diversification in the search process, which in turn enables the attainment of high-quality solutions within reasonable computational times.

According to Öztaş and Tuş (2022), the strength of ILS lies in its ability to combine a global exploration of the solution space with an intensive exploitation of promising regions, without requiring high algorithmic complexity. Moreover, its modular structure facilitates integration with other metaheuristic strategies, such as variable neighborhood search or hybrid techniques, thus broadening its applicability to logistical, routing, and other optimization problems. In summary, Iterated Local Search has established itself as a powerful and versatile tool for successfully tackling large-scale problems with complex constraints.

Due to the simplicity of this algorithm, it was later rediscovered by several authors, including Baum(1986).

Algorithm 4 presents the pseudo-code of the ILS-VND proposed for the SCMVRPPD. The constant value $maxIter$ is the number of iterations of the main loop. In each iteration, an initial solution is generated. The initial solution is improved using the $RandomMixShift$ procedure, and the $iterILS$ variable is fixed to 0. The number of iterations of the inner loop is limited by another constant, $MaxIterILS$. Within the inner loop, the local search procedure (VND) is performed, generating a solution s. If the solution s is better than the current solution s', then s' is updated with s, s is replaced with a perturbation (see Section 4.5) of s', and the number of iterations $iterILS$ is reset to 0. Otherwise, $iterILS$ is increased by one. We always keep s^*, the best solution found so far.

Algorithm 4: ILS-VND.

```
for k := 1, ..., MaxIter do
    s := GenerateInitialSolution(seed);
    s' := RandomMixShift(s) ;
    iterILS := 0;
    f(s) denotes the cost function of a solution s. ;
    while iterILS < MaxIterILS do
        r = number of neighborhoods. ;
        s := VND(N(.), r, s) ;
        if f(s) < f(s') then
            s' := s ;
            s := Perturb(s') ;
            iterILS := 0 ;
        else
            iterILS := iterILS + 1 ;
    if f(s') < f(s*) then
        s* := s' ;
```

4.8. VNS

Variable Neighborhood Search (VNS) is a metaheuristic proposed by Mladenović and Hansen (1997). The idea is to apply a perturbation to the search operator when a local minimum is reached. This perturbation reactivates the search to reach a solution that cannot be found with the current search operator, allowing further exploration of the solution space. VNS generally consists of three main steps: shaking, local search, and neighborhood change. The local search phase is designed to identify a local optimum within a given neighborhood, while the shaking phase serves to escape that local optimum and create a new starting point for subsequent local search. Several enhancements of the original method have been proposed (e.g., see Azzahra and Santosa (2021), Kyriakakis et al. (2022), Sadati and Çatay (2021), Sun et al. (2023)).

Algorithm 5 presents the pseudo-code of our VNS for the SCMVRPPD. The algorithm begins by initializing the parameters *diversification* and *MaxIter*. An initial solution s is generated by the *FindInitialSolution* heuristic. The Random Mix-Shift heuristic is then used to refine the initial solution. The main loop iterates until the maximum number of runs *MaxIter* is reached. A perturbation procedure called *Shake* is applied to the current solution s_i, generating a solution s_i'. If s_i' has a lower cost than the best solution s^*, then s^* is updated with s_i', and the iteration counter $iter$ is reset to 1. The local search (VND) is then performed on s_i', generating a solution s_k. If s_k is better than s^*, we update s^* with s_k and reset $iter$ to 1. If the local search does not improve s^*, we use a method called *AcceptanceCriteria* to accept s_k as the new current solution if $f(s_k) < \beta f(s^*)$. Otherwise, it returns s^*. The idea is not to deviate too far from a locally optimal solution.

Finally, the diversification phase is carried out once during the algorithm's execution. If $iter = MaxIter$ and diversification $= 0$, $iter$ is reset to 0, diversification is set to 1, and a perturbation (*Shake*) is applied to the current solution s_i.

Algorithm 5: GVNS Heuristic.

```
s := FindInitialSolution(seed);
s_i := RandomMixShift(s) ;
diversification := 0;
for iter := 1 to MaxIter do
    perturbation phase ;
    s_i' := Shake(s_i) ;
    f(s) denotes the cost function of a solution s. ;
    if f(s_i') < f(s*) then
        s* := s_i';
        iter := 1;
    s_k := VND(s_i');
    if f(s_k) < f(s*) then
        s* := s_k;
        iter := 1;
    else
        s_k :=AcceptanceCriteria(s*, s_k);
    s_i := s_k;
    diversification phase ;
    if iter = MaxIter and diversification = 0 then
        iter := 0;
        diversification := 1;
        s_i := Shake(s_i);
```

5. Tabu search algorithm

We present an adaptation of the TABUSTOCH heuristic proposed by Gendreau et al. (1996), originally designed for the Vehicle Routing Problem (VRP) with stochastic demands. The algorithm solves a two-stage stochastic VRP: in the first stage, a feasible solution is constructed by including all vertices; in the second stage, recourse actions may be taken once the real demands of customers are revealed and capacity constraints might be violated. When traversing a route, if a vehicle becomes full, we apply Strategy (a) (see Section 3.2) to free space in the vehicle. All parameters in the adapted algorithm remain the same as in the original TABUSTOCH. Below, we present only the modifications made to TABUSTOCH to handle the SCMVRPPD.

Let x^k be a first-stage solution in iteration k of the algorithm. We then compute $T(x^k)$, which is the expected value of the solution when considering the second stage. We define $T(x^k) = \sum_{i=1}^{m^k} T^i(x^k)$, where m^k is the number of routes at iteration k, and $T^i(x^k)$ is the expected cost of route i in solution x^k.

The initial solution is built by assigning to each depot the closest *candidate* pair of pickup and delivery. A pickup-delivery pair is considered *candidate* if it has not yet been assigned to any depot. The chosen pickup-delivery pair is appended to the solution (first the pickup, then the delivery). The initial solution is always feasible. The neighborhood structure used by TABUSTOCH is the Mix-Shift(1,0) operator presented in Section 4.4, thus allowing the possibility of inserting pairs of pickup and delivery points into different cycles.

We treat vertex movements in a solution as elements of the tabu list. There are two ways to move a vertex: 1) change the position of the vertex within the same tour, and 2) move the vertex (along with its corresponding pair) to another tour.

Either of these two movements is tabu for θ iterations, where θ is randomly selected from the interval $[|V|-5, |V|]$. In each iteration, the search for solutions considers the current solution x^k and selects the best non-tabu solution x^{k+1} in the Mix-Shift(1,0) neighborhood. However, a tabu solution can be chosen if it improves the current best solution T^* (aspiration criterion). To accelerate the search process, we use simple approximations of insertion cost (see Eq. (10)). Specifically, A_2 is used to approximate the change in solution cost when performing a move. Nevertheless, once the best neighboring solution is found, we compute its exact expected cost to verify whether it improves the current best solution.

6. Computational experiments

We conducted computational experiments to evaluate the performance of the ILS-VND, GVNS, and TS algorithms. The algorithms were implemented in C++. All experiments were run on a Linux operating system with 3 GB of memory and an Intel Core i5 2.54×4 GHz processor. Ten independent runs of each algorithm were performed for every test case. The SCMVRPPD instances are described in Section 6.1. The configuration and parameter tuning of the algorithms are described in Section 6.2. Finally, the comparative analysis and the overall performance of the algorithms are presented in Sections 6.3 and 6.4, respectively.

6.1. Test instances

Our experiments were conducted on instances derived from those presented in Haddad et al. (2018) for the multi-vehicle pickup-and-delivery problem with split deliveries. Throughout this section, we refer to the instances in Haddad et al. (2018) as the original instances.

The instances proposed in this work were generated so that the capacity of a vehicle is always 2 or 5 units. If the capacity is very large (for example, greater than the number of pickup points in the route), unloading would not be necessary. On the other hand, if the vehicle's capacity were only 1 unit, the vehicle would have to use an unloading strategy every time it reached a pickup point. Both of these situations are less common. We therefore focus on the more typical case of delivery vehicles (often motorcycles) with capacities of 2 or 5 units. These small capacities prevent either excessive or nonexistent use of unloading strategies.

The depots were randomly selected from the set of vertices in the original instance. The number of depots depends on whether the original instance has an even or odd number of vertices. If the number of vertices in the original instance is even, we randomly select either 2 or 4 vertices to serve as depots. If the number of vertices is odd, we randomly select 3 or 5. Since the SCMVRPPD considers multiple depots, we define $|D|$ within the interval $[2, 5]$. For the remaining vertices (those not selected as depots), we performed a random matching to create pickup-delivery pairs. The probability of presence of each pickup-delivery pair follows a uniform distribution in $[0, 1]$ and was independently selected for each pair.

Four new instances were generated from each original instance. For example, *mpdpsl20-1a*, *mpdpsl20-1b*, *mpdpsl20-1c*, and *mpdpsl20-1d* were generated from the original instance *mpdpsl20-1*. These four new instances represent all possible combinations of depot numbers and capacities. We considered a diverse set of instances ranging from 20 to 250 vertices, resulting in 296 new instances in total. The dataset and a description of each instance are available at https://www.loco.ic.unicamp.br/files/instances/, and we refer to this dataset as SCMDVRPPLib.

6.2. Calibration of the algorithms

As is common in many metaheuristics, the ILS-VND, GVNS, and TS algorithms rely on several parameters and configuration options for search operators. To identify good parameter values, we used the IRACE tool (López-Ibáñez, Dubois-Lacoste, Cáceres, Birattari, & Stützle, 2016), which has proven effective for calibrating various metaheuristics. IRACE takes as input a parameter space corresponding to the parameters to be tuned, a set of training instances on which to calibrate the parameters, and a set of IRACE options that define the configuration scenario (e.g., the maximum number of iterations before the tool stops). IRACE then searches for the best algorithmic settings in the parameter space by running each algorithm on different instances with varying parameter configurations. The training instances for the algorithms are listed in Table 3. The goal is to find a set of parameter values that works well for all instances considered.

In IRACE, each parameter is associated with a domain of a particular type (integer, real, categorical, or ordinal). The parameters used by the algorithms were of type integer or real. Table 2 presents the final parameter settings recommended by IRACE for each algorithm, trained with the instances in Table 3. The table has three columns for each algorithm: the parameter name, the range of parameter values explored, and the values IRACE suggested. Note that ILS-VND and GVNS share the same parameters, but we consider different intervals due to the iterative nature of each algorithm. To determine the lower and upper limits of each parameter's range, we carried out empirical experiments. For example, increasing the value of *MaxIterShif* in the GVNS algorithm to 100 yields no better results than using a value of 50, but simply requires more computation time. Similarly, decreasing *MaxIterShif* below 10 terminates the algorithm too quickly, preventing it from escaping local optima.

6.3. Comparative analysis

We use performance profiles to compare the results of the algorithms. Performance profiles provide a cumulative distribution of each algorithm's performance relative to the others (Dolan & Moré, 2002). This comparison method involves, for each instance, calculating the ratio between a performance metric (e.g., cost) for that instance and the best result achieved among all algorithms. Thus, if an algorithm has a ratio of 1 for a particular instance, it means that it achieved the best result for that instance among all algorithms.

Fig. 14 shows five subplots comparing the performance of ILS-VND, GVNS, and TS from different perspectives. Panel (**a**) displays the performance profile of the best cost (*best cost*) for all three algorithms, where the horizontal axis (f) indicates how close each algorithm's solution is to the global best cost, and the vertical axis indicates the fraction of instances that meet this criterion. Panel (**b**) repeats the best-cost performance profile but only for GVNS and TS, excluding ILS-VND (already identified as the overall best performer). Panel (**c**) shows the performance profile based on the average of the best costs obtained in the ten independent executions for each instance, again for all three algorithms. Panel (**d**) provides the same analysis as (c) but focuses only on GVNS and TS. Finally, panel (**e**) compares the fraction of instances solved over time, illustrating each algorithm's computational efficiency.

For instance, in the top-left subplot of Fig. 14, TS has the point $(1, 0.63)$, meaning that it achieved the best cost in 63 % of the instances. It also has the point $(1.75, 0.95)$, meaning that it solved 95 % of the instances with a cost at most 1.75 times the best cost. We computed

Table 2
Parameters tuned by IRACE for ILS-VND, GVNS, and TS.

ILS			GVNS		
Param.	Interval	Value	Param.	Interval	Value
MaxIterShif	[10,70]	50	MaxIterShif	[10,50]	48
alpha	[1,1.5]	1.03	alpha	[1.0,2.0]	1.33
MaxIter	[10,20]	18	MaxIter	[100,500]	362
MaxIterILS	[10,20]	19	MaxIterILS	[1.0,2.0]	1.15

TS		
Param.	Interval	Value
p	[5,15]	10
q	[5,10]	10
t	[1,50]	31

Table 3
Training instances.

Name	$\|V\|$	$\|D\|$	Q	Name	$\|V\|$	$\|D\|$	Q
mpdpsl20-1a.vrppd	20	2	2	mpdpsl50-4c.vrppd	50	4	2
mpdpsl20-4c.vrppd	20	4	2	mpdpsl50-9a.vrppd	50	2	2
mpdpsl20-8c.vrppd	20	4	2	Nowak150-1Bb.vrppd	150	2	5
mpdpsl30-5b.vrppd	30	2	5	Nowak150-2Ab.vrppd	150	2	5
mpdpsl30-8d.vrppd	30	4	5	Nowak150-2Cc.vrppd	150	4	2
mpdpsl30-10b.vrppd	30	2	5	Nowak200-1Ac.vrppd	200	4	2
mpdpsl40-2c.vrppd	40	4	2	Nowak200-2Ba.vrppd	200	2	2
mpdpsl40-8c.vrppd	40	4	2	Nowak200-2Ed.vrppd	200	4	5
mpdpsl40-9d.vrppd	40	4	5	Nowak250-1Cc.vrppd	250	4	2
mpdpsl50-2a.vrppd	50	2	2				

all such ratios for all instances, taking into account both solution cost and execution time. The results show that ILS-VND found the best solution costs in nearly 100 % of the instances. GVNS and TS are close to each other, but TS performs slightly better than GVNS. When we consider the average solution cost among the 10 independent runs, ILS-VND still finds the best solutions, with TS performing better than GVNS. In terms of runtime, GVNS is the fastest algorithm, solving all instances in at most 300 s, whereas TS and ILS-VND have similar performance times.

Regarding instance size, we classify instances into three types: small (20 to 50 vertices), medium (50 to 150 vertices), and large (200 to 250 vertices). For small instances, the average results are very similar among the three algorithms, with GVNS standing out because it is the fastest. In medium instances (50 to 150 vertices), TS, GVNS, and ILS-VND yield similar results, although ILS-VND appears to outperform the others in instances with more than 100 vertices. Finally, for large instances (more than 200 vertices), ILS-VND attains a significant advantage over TS and GVNS, consistently finding the best solutions. TS and GVNS exhibit similar performance for large instances, mirroring their behavior on medium ones. With respect to the number of depots in the network, we do not see major differences in performance, likely because the variation in depot numbers across the instances is not large. Similarly, we do not observe significant differences when varying the vehicle capacities.

6.4. Analysis of overall performance

In analyzing the algorithms' overall performance, we used the average values they achieved in each instance for comparison. We applied the Friedman test to check for significant differences between the solutions obtained by the algorithms. For interpretation, H_0 (the null hypothesis) states that the samples come from the same population, and H_a (the alternative hypothesis) states that they do not. We used a significance level of $\alpha = 0.05$. If the calculated p-value is less than α, we reject H_0 and accept H_a. A significant Friedman test indicates that at least two of the compared groups differ significantly, but it does not specify which groups. To determine which specific groups differ, we performed a post-hoc multiple-comparison test. This approach involves adjusting the significance level α to avoid inflating Type I error (i.e., rejecting a true null hypothesis). The Nemenyi procedure was used as the post-hoc test. We used XLSTAT for statistical analysis of the algorithm results and for applying the Friedman test.

Table 4
p-Values for pairwise comparisons among the algorithms.

	ILS-VND	GVNS	TS
ILS-VND	1	**<0.0001**	**<0.0001**
GVNS	**<0.0001**	1	**0.002**
TS	**<0.0001**	**0.002**	1

After running the experiments, the Friedman test showed significant differences among the average solution costs of ILS-VND, GVNS, and TS, with a p-value less than 0.0001. Since this p-value is below the significance level $\alpha = 0.05$, the null hypothesis H_0 is rejected, and the alternative hypothesis H_a is accepted. The Nemenyi procedure identified three groups of algorithms, each group containing algorithms that do not differ statistically from one another. According to the Nemenyi procedure, all the algorithms belong to different groups, indicating a statistical difference between each pair. Table 4 shows the p-values for pairs of algorithms, all of which are below 0.05, confirming the significance of these differences.

7. Future work and opportunities

The results obtained in this study have demonstrated the effectiveness of the approaches implemented to address the SCMVRPPD. However, there are still numerous opportunities to improve and expand research in this field. One of the most promising directions lies in the **creation of more representative and diverse datasets** that more accurately reflect real-world conditions. To achieve this, instances can

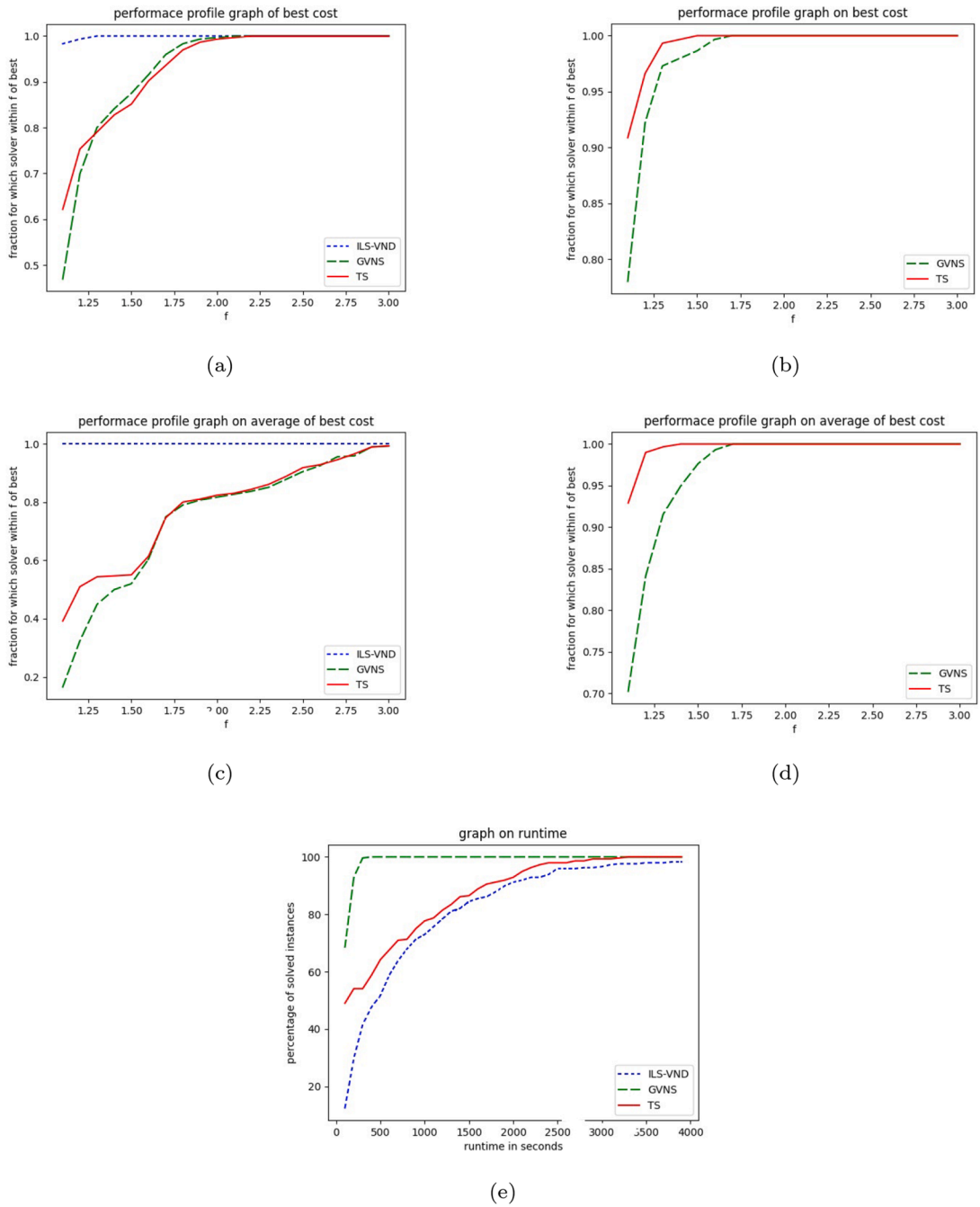

Fig. 14. Performance profiles on best cost, average cost, and runtime for the TS (red line), GVNS (green line), and ILS-VND (blue line) algorithms. In these plots, f is a real-valued factor of the best possible performance ratio. See Dolan and Moré (2002) for more details.

be generated that incorporate fluctuations in demand, geographical constraints, weather conditions, and delivery schedules. Additionally, the application of simulation techniques and synthetic data generation could significantly enrich the evaluation and testing scenarios for the algorithms.

A crucial aspect is the **exploration of hybrid approaches** that integrate different algorithmic strategies, combining metaheuristics with exact methods. This synergy would enhance the quality of solutions and optimize computational efficiency. In particular, using exact methods in the early stages of the process could provide more refined base solutions to feed metaheuristics, reducing the search space and increasing the convergence rate toward optimal solutions. Furthermore, it would be interesting to investigate adaptive strategies that dynamically adjust algorithm parameters based on problem complexity and solution evolution.

The **analysis of real-time applications** represents another highly relevant area, as demand in logistics environments is highly dynamic and requires quick and efficient responses. Implementing systems that incorporate real-time monitoring techniques and machine learning could improve operational decision-making, allowing for continuous route updates and optimization of delivery times. Additionally, studying the **impact of uncertainty** on solution quality would enable the development of more robust strategies to handle unforeseen variations in environmental conditions. In this regard, the design of probabilistic models or robust optimization approaches could contribute to improving solution stability in highly volatile scenarios.

In terms of **scalability**, evaluating the performance of algorithms on larger and more complex instances is essential to ensure their applicability in real-world situations. Computational efficiency becomes a determining factor as the problem size increases, making it crucial to analyze processing time, memory consumption, and the algorithms' ability to respond within a reasonable timeframe in large-scale environments. Optimizing data structures and exploring parallelization techniques could be key strategies to enhance scalability and ensure optimal performance across different operational contexts.

We consider it valuable to incorporate online customer requests and transform SCMVRPPD into a dynamic and stochastic problem. At the same time, studying exact methods for the optimal resolution of small and medium-sized instances would provide reference solutions to evaluate the performance of heuristic and metaheuristic approaches. The application of stochastic integer programming models and the exploration of techniques such as L-shaped decomposition could provide powerful tools to address these problem variants with greater precision.

Regarding future research directions, we recommend deepening the application of **artificial intelligence and machine learning** techniques to refine customer demand forecasting and optimize route planning. These methodologies can significantly contribute to more informed decision-making, reducing uncertainty and improving operational efficiency. Additionally, incorporating further **operational constraints**, such as time windows for deliveries, regulations on driving hours, and vehicle maintenance periods, would increase the model's applicability in real-world scenarios. Another key area of study is analyzing how different order **distribution patterns** affect algorithm performance, thereby enabling the design of more adaptable and efficient strategies for various logistical configurations. Finally, advancing the development of more e**fficient and robust metaheuristics** tailored to the specific characteristics of SCMVRPPD is necessary. Addressing these improvement opportunities and continuing research in these directions will significantly drive progress in the field of vehicle routing, providing innovative solutions to current and future logistical challenges.

8. Discussion and implications

The formulation, implementation, and validation of the SCMDVRPPD provide important insights into the complexity of routing problems and the strategies for addressing them in stochastic environments. Several key aspects emerge when contrasting the experimental results with the theoretical proposals and the state of the art in the literature.

First, it is worth emphasizing the combinatorial richness inherent in the SCMDVRPPD. The inclusion of multiple depots, paired pickups and deliveries, capacity constraints, and the stochastic dimension makes the problem strongly NP-hard. Even so, the proposed metaheuristics (ILS-VND, GVNS, TS) have been able to handle medium- to large-scale instances with promising results. The performance of ILS-VND, in particular, highlights the potential of combining systematic local searches, such as VND, with iterative methods that encourage diversification. This confirms the importance of balancing intensification and diversification when designing algorithms for complex routing problems.

Comparisons with the existing literature are another relevant factor in this discussion. Many previous studies address subproblems or related issues such as the Multi-Depot VRP, the Pickup and Delivery Problem, or the Vehicle Routing Problem under various stochastic approaches. However, most do not simultaneously consider all the elements (multiple depots, limited capacity, paired pickups and deliveries, stochastic demand). SCMDVRPPD underscores the need for hybrid and flexible approaches that can adapt to realistic scenarios in which demand is not guaranteed and capacity is not unlimited. Thus, it reinforces the line of thought that pure heuristics or classic methodologies (for example, simple branch-and-bound) can hardly scale when so many constraints are combined.

On the other hand, the inclusion of capacity in a stochastic context has been one of this study's most noteworthy contributions. The fact that vehicles can fill up and "fail" halfway through a route, generating additional costs (penalties or adjustments), underlines the need for effective strategies to handle uncertainty. Among the three correction strategies proposed, the decision was ultimately to rely on a penalty-based approach (outsourcing the delivery), given its computational simplicity, although it may introduce extra costs in high-density demand scenarios. This operational decision is justified by the method's robustness and its clear advantage in computing the expected cost of routes a priori. However, in environments where the penalty is excessively costly, the other two strategies (partial or total unloading) might be reconsidered with more advanced analytical and heuristic models.

From a methodological standpoint, it is worth highlighting the relevance of the statistical tests employed (Friedman test and post-hoc tests). In SCMDVRPPD, the resulting values (e.g., total routing costs, computation times, solution quality) often do not follow normal distributions. Moreover, the behavior of each algorithm can vary depending on the test instance, the random seed, or the topology of the client and depot network. Therefore, a statistical approach is required that does not rely on strong assumptions about normality or homoscedasticity. Given these conditions of variability, the Friedman test becomes one of the most appropriate non-parametric techniques to compare three or more algorithms over multiple instances. Unlike traditional ANOVA, Friedman is based on rank analysis assigned to each method in each dataset, thus not requiring normality or homogeneous variances. After applying Friedman, it is common to use post-hoc procedures (Nemenyi, Holm, or Bonferroni) to identify which algorithms differ significantly from each other. This makes it possible to determine, for example, whether ILS-VND statistically outperforms GVNS and TS or whether there is a difference between these two.

Additionally, other non-parametric tests can be considered for future work:

(a) Quade's Test: similar to Friedman, but with weighting based on instance difficulty.
(b) Iman-Davenport's Test: enhances statistical power for a large number of instances.
(c) Bayesian Tests: a recent approach offering direct interpretations regarding one method's superiority over another.

In summary, integrating non-parametric tests, with a special emphasis on Friedman, constitutes a fundamental pillar to support conclusions about the relative superiority of the analyzed metaheuristics. This methodology increases the reliability of the findings by showing that the differences observed among ILS-VND, GVNS, and TS are statistically significant and not merely coincidental. It also opens the door to applying other robust tests in the future –such as Quade, Iman-Davenport, or Bayesian comparisons– that could further refine the performance assessment of each method.

Finally, as a general reflection, the SCMDVRPPD illustrates a clear trend in the optimization research community and, specifically, in vehicle routing: advancing toward models with uncertain components and multiple constraints that more closely resemble real-world logistic problems. The discussion in this study demonstrates that it is feasible to tackle these complexities with modern, detailed metaheuristics, provided there is adequate computational infrastructure and robust statistical methodologies to evaluate the results.

9. Conclusion

This study has presented a comprehensive approach to addressing the SCMVRPPD problem using advanced metaheuristics, including ILS-VND, GVNS, and TS. One of them is a pure metaheuristic (TS), while

the others represent a more complex hybridization, incorporating VND as part of the algorithm's local search mechanism.

In terms of solution quality for the hybrid algorithms, the best results were obtained by ILS-VND, followed by GVNS. The experiments demonstrated the efficiency of ILS-VND, which achieved superior solutions. In this regard, the results are considered highly promising, as they highlight the importance of developing hybrid approaches to solve the problem. The ILS-VND algorithm achieved the expected results by leveraging the power of VND local searches, obtaining the best performance among all the investigated algorithms. The GVNS algorithm, although less competitive than ILS-VND, demonstrated robust performance in specific scenarios, while TS, despite being efficient in execution time, was outperformed in solution quality by the other two strategies. The proposed algorithms are highly promising, as significant improvements were achieved in almost all evaluated instances.

One of the key findings of the study is that hybrid strategies offer a significant advantage over purely metaheuristic methodologies. The application of the Friedman Test has confirmed statistically significant differences between the evaluated algorithms, reinforcing the need to further explore hybrid approaches.

Regarding the applicability of the study, the obtained results have direct relevance to logistics optimization and transportation planning, particularly in scenarios where demand is uncertain and fluctuating. The implementation of robust optimization strategies could significantly contribute to reducing operational costs and improving distribution efficiency in large-scale logistics networks.

In conclusion, this study has significantly contributed to the field of vehicle routing, highlighting the importance of hybrid approaches and providing a solid framework for future research in logistics and transportation optimization. The exploration of more adaptive strategies, combined with artificial intelligence and robust optimization, will enable the development of even more efficient and scalable solutions for current and future challenges in transportation management.

CRediT authorship contribution statement

Brenner Humberto Ojeda Rios: Methodology, Project administration, Writing- Reviewing and Editing, Writing- Original draft preparation, Formal analysis. **Eduardo Xavier:** Conceptualization, Methodology, Supervision, Investigation, Formal analysis

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Brenner Humberto Ojeda Rios reports financial support was provided by University of Engineering and Technology. Brenner Humberto Ojeda Rios reports that financial support, statistical analysis, and writing assistance were provided by the University of Engineering and Technology. Eduardo Xavier reports having a relationship with MercadoLivre.com Atividades de Internet Ltda that includes employment. The authors declare that there is no conflict of interest, nor any additional relationships or activities that could be interpreted as influencing the work presented. This study has been conducted strictly for academic purposes, without any financial, personal, or professional interests that could bias or affect the results. Therefore, no conflict of interest is identified for the reader. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have influenced the work reported in this article. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Strategies

To compute the expected cost $E[l_s]$ of a route s we use Algorithm 6. Let s_i be a valid route generated from the route s, e.g., suppose that $s_1 = (v_0)$ and $s_2 = (v_0, r_1, c_1, v_0)$ are valid routes. In s_1 no pair of pickup and delivery appear in the route. In s_2 only the pair (r_1, c_1) appears. S denotes the set of valid routes generated from s. There are 2^q valid routes generated from s.

Algorithm 6: Generic brute force algorithm to calculate expected cost.

```
exp = 0 ;
for s_i ⊆ S do
    prob := 1, cost := Cost of the route s_i using some strategy
    for (r_j, c_j) ∈ s_i do
        prob = prob × P(r_j)
    for (r_j, c_j) ∉ s_i do
        prob = prob × (1 − P(r_j))
    exp = exp + cost × prob
```

A.1. Sensitivity and Robustness Analysis of the Strategies

The sensitivity analysis presented in this study, based on Table A.5, focuses on systematically evaluating how the results of strategies (a), (b), and (c) vary in response to changes in the key parameters of the problem. In particular, critical variables such as vehicle capacity, the number of depots, and the cost associated with penalties are examined. Table A.5 presents the results obtained using a brute-force approach to evaluate the performance of strategies (a), (b), and (c), along with the outcomes of the algorithm implementing the theorem developed in Section 3.3 for strategy (a). This table provides a comparative analysis that validates the accuracy of the theoretical method against an exhaustive exploration of all possible configurations. The data presented demonstrate both the computational efficiency of each strategy and the robustness of the theorem-based algorithm, providing a solid foundation for selecting and tuning the optimal methodology for route correction.

The results indicate that the correlation between vehicle capacity and the expected cost is $r = 0.091$, suggesting that variations in capacity have only a marginal impact on the total cost. In contrast, the probability of an order being placed shows a moderate correlation of $r = 0.387$, demonstrating that increases in this probability lead to a noticeable rise in the expected cost due to a higher frequency of route executions. Similarly, the number of depots exhibits a minimal correlation with the cost ($r = 0.050$), indicating that, within the analyzed range, this variable does not significantly affect the system's performance. Finally, the cost associated with penalties shows a negative correlation of $r = -0.098$, suggesting that strategies incorporating high penalties may induce route adjustments that offset the increase in these costs.

Additionally, the comparison between the brute-force approach and the algorithm based on the theorem developed in Section 3.3 for strategy (a) reveals a discrepancy of less than 2% in the expected cost, thereby validating the accuracy and robustness of the proposed theoretical method. These findings confirm the stability of the implemented strategies and provide a reliable tool for optimizing route correction in scenarios characterized by high operational uncertainty.

Table A.5
Results of the brute force algorithms for strategies (a)-(b) and of the algorithm that uses the theorem presented in Section 3.3 for Strategy (a).

Instance				Strategy A		Strategy B		Strategy C		Strategy A with theorem	
Name	N	D	Q	E	Time	E	Time	E	Time	E	Time
mpdpsl20-1a	20	2	2	334.691	0.0000	303.349	0.0010	294.32	0.0010	334.691	0.0000
mpdpsl20-1b	20	2	5	315.534	0.0000	314.731	0.0040	314.699	0.0030	315.534	0.0000
mpdpsl20-1c	20	4	2	519.553	0.0000	416.169	0.0000	415.132	0.0000	519.553	0.0000
mpdpsl20-1d	20	4	5	510.978	0.0000	510.978	0.0000	510.978	0.0000	510.978	0.0000
mpdpsl20-2a	20	2	2	830.132	0.0000	507.945	0.0030	523.75	0.0040	830.132	0.0010
mpdpsl20-2b	20	2	5	407.267	0.0000	407.747	0.0020	406.369	0.0010	407.267	0.0000
mpdpsl20-2c	20	4	2	620.327	0.0000	577.785	0.0000	577.785	0.0000	620.327	0.0000
mpdpsl20-2d	20	4	5	345.236	0.0000	345.236	0.0000	345.236	0.0000	345.236	0.0000
mpdpsl20-3a	20	2	2	543.342	0.0000	414.804	0.0010	407.844	0.0010	543.342	0.0000
mpdpsl20-3b	20	2	5	388.106	0.0000	388.106	0.0000	388.106	0.0000	388.106	0.0000
mpdpsl20-3c	20	4	2	557.991	0.0000	445.896	0.0000	447.375	0.0000	557.991	0.0000
mpdpsl20-3d	20	4	5	338.086	0.0000	338.086	0.0000	338.086	0.0000	338.086	0.0000
mpdpsl20-4a	20	2	2	763.98	0.0000	473.417	0.0020	499.497	0.0020	763.98	0.0000
mpdpsl20-4b	20	2	5	457.121	0.0000	457.121	0.0000	457.121	0.0010	457.121	0.0000
mpdpsl20-4c	20	4	2	552.633	0.0000	408.842	0.0000	405.435	0.0000	552.633	0.0000
mpdpsl20-4d	20	4	5	437.254	0.0000	437.254	0.0010	437.254	0.0000	437.254	0.0000
mpdpsl20-5a	20	2	2	537.259	0.0000	424.13	0.0000	416.348	0.0000	537.259	0.0000
mpdpsl20-5b	20	2	5	448.572	0.0000	448.572	0.0010	448.572	0.0000	448.572	0.0000
mpdpsl20-5c	20	4	2	389.796	0.0010	381.078	0.0000	381.061	0.0000	389.796	0.0000
mpdpsl20-5d	20	4	5	418.834	0.0000	418.834	0.0000	418.834	0.0000	418.834	0.0000
mpdpsl20-6a	20	2	2	539.523	0.0000	345.353	0.0030	367.731	0.0040	539.523	0.0000
mpdpsl20-6b	20	2	5	413.939	0.0000	413.939	0.0000	413.939	0.0010	413.939	0.0000
mpdpsl20-6c	20	4	2	324.154	0.0000	230.991	0.0010	229.135	0.0000	324.154	0.0000
mpdpsl20-6d	20	4	5	419.404	0.0000	419.404	0.0000	419.404	0.0010	419.404	0.0000
mpdpsl20-7a	20	2	2	758.696	0.0000	513.407	0.0020	494.72	0.0020	758.696	0.0000
mpdpsl20-7b	20	2	5	413.762	0.0000	413.762	0.0000	413.762	0.0010	413.762	0.0000
mpdpsl20-7c	20	4	2	562.788	0.0000	426.255	0.0000	413.553	0.0000	562.788	0.0000
mpdpsl20-7d	20	4	5	462.907	0.0000	462.907	0.0000	462.907	0.0000	462.907	0.0000
mpdpsl20-8a	20	2	2	533.478	0.0000	394.582	0.0030	411.057	0.0040	533.478	0.0000
mpdpsl20-8b	20	2	5	411.097	0.0000	410.392	0.0010	410.293	0.0010	411.097	0.0000
mpdpsl20-8c	20	4	2	506.665	0.0000	486.385	0.0010	486.385	0.0000	506.665	0.0000
mpdpsl20-8d	20	4	5	426.474	0.0010	426.474	0.0000	426.474	0.0000	426.474	0.0000
mpdpsl20-9a	20	2	2	518.996	0.0010	385.069	0.0030	420.28	0.0040	518.996	0.0000
mpdpsl20-9b	20	2	5	226.897	0.0000	225.699	0.0030	225.244	0.0030	226.897	0.0000
mpdpsl20-9c	20	4	2	464.825	0.0000	383.835	0.0010	383.318	0.0000	464.825	0.0000
mpdpsl20-9d	20	4	5	565.403	0.0000	565.403	0.0010	565.403	0.0000	565.403	0.0000
mpdpsl20-10a	20	2	2	708.616	0.0000	590.428	0.0020	542.197	0.0010	708.616	0.0000
mpdpsl20-10b	20	2	5	493.66	0.0000	493.66	0.0010	493.66	0.0000	493.66	0.0010
mpdpsl20-10c	20	4	2	650.948	0.0000	571.283	0.0000	571.283	0.0000	650.948	0.0000
mpdpsl20-10d	20	4	5	395.109	0.0000	395.109	0.0000	395.109	0.0000	395.109	0.0000
mpdpsl30-1a	30	2	2	1174.96	0.0080	754.107	0.2080	770.682	0.2050	1174.96	0.0000
mpdpsl30-1b	30	2	5	725.729	0.0010	702.681	0.0160	686.06	0.0150	725.729	0.0000
mpdpsl30-1c	30	4	2	496.155	0.0000	454.286	0.0020	452.61	0.0020	496.155	0.0010
mpdpsl30-1d	30	4	5	659.187	0.0010	602.087	0.0030	591.632	0.0030	659.187	0.0000
mpdpsl30-2a	30	2	2	1149.17	0.0010	730.497	0.0080	722.291	0.0090	1149.17	0.0000
mpdpsl30-2b	30	2	5	563.375	0.0000	555.148	0.0030	554.796	0.0030	563.375	0.0000
mpdpsl30-2c	30	4	2	751.406	0.0000	640.754	0.0000	635.647	0.0010	751.406	0.0000
mpdpsl30-2d	30	4	5	519.177	0.0000	519.177	0.0020	519.177	0.0010	519.177	0.0000
mpdpsl30-3a	30	2	2	1052.41	0.0010	706.515	0.0190	764.114	0.0200	1052.41	0.0010
mpdpsl30-3b	30	2	5	492.35	0.0000	492.35	0.0040	492.35	0.0030	492.35	0.0000
mpdpsl30-3c	30	4	2	853.487	0.0000	717.022	0.0000	713.023	0.0010	853.487	0.0000
mpdpsl30-3d	30	4	5	588.409	0.0000	588.339	0.0010	588.322	0.0010	588.409	0.0000
mpdpsl30-4a	30	2	2	1562.55	0.0040	1028.87	0.0910	1087.72	0.0910	1562.55	0.0000
mpdpsl30-4b	30	2	5	625.925	0.0010	635.818	0.0030	607.972	0.0040	625.925	0.0000
mpdpsl30-4c	30	4	2	712.602	0.0000	574.033	0.0040	547.761	0.0030	712.602	0.0000
mpdpsl30-4d	30	4	5	644.835	0.0000	644.835	0.0010	644.835	0.0020	644.835	0.0000
mpdpsl30-5a	30	2	2	894.952	0.0010	629.337	0.0040	625.818	0.0040	894.952	0.0000
mpdpsl30-5b	30	2	5	592.139	0.0010	588.639	0.0030	587.864	0.0040	592.139	0.0010
mpdpsl30-5c	30	4	2	885.927	0.0000	669.084	0.0010	655.569	0.0020	885.927	0.0000
mpdpsl30-5d	30	4	5	467.804	0.0000	467.804	0.0000	467.804	0.0000	467.804	0.0000
mpdpsl30-6a	30	2	2	1017.59	0.0020	648.458	0.0430	639.547	0.0430	1017.59	0.0010
mpdpsl30-6b	30	2	5	615.862	0.0000	615.859	0.0030	615.857	0.0020	615.862	0.0000
mpdpsl30-6c	30	4	2	1019.62	0.0000	733.959	0.0010	741.402	0.0010	1019.62	0.0000
mpdpsl30-6d	30	4	5	509.401	0.0000	509.401	0.0030	509.401	0.0030	509.401	0.0000
mpdpsl30-7a	30	2	2	990.973	0.0000	701.247	0.0080	682.534	0.0080	990.973	0.0000
mpdpsl30-7b	30	2	5	480.39	0.0040	473.869	0.0780	460.008	0.0760	480.39	0.0000
mpdpsl30-7c	30	4	2	931.973	0.0000	711.901	0.0010	662.842	0.0010	931.973	0.0000
mpdpsl30-7d	30	4	5	416.931	0.0000	416.931	0.0030	416.931	0.0030	416.931	0.0000
mpdpsl30-8a	30	2	2	1050.57	0.0090	692.353	0.2160	721.115	0.2170	1050.57	0.0010

Table A.5
(*Continued*).

Instance				Strategy A		Strategy B		Strategy C		Strategy A with theorem	
Name	N	D	Q	E	Time	E	Time	E	Time	E	Time
mpdpsl30-8b	30	2	5	695.771	0.0080	655.723	0.1720	620.701	0.1670	695.771	0.0000
mpdpsl30-8c	30	4	2	979.364	0.0000	744.629	0.0000	759.489	0.0010	979.364	0.0000
mpdpsl30-8d	30	4	5	411.158	0.0000	411.158	0.0010	411.158	0.0020	411.158	0.0000
mpdpsl30-10a	30	2	2	974.145	0.0090	634.362	0.1990	614.195	0.1970	974.145	0.0000
mpdpsl30-10b	30	2	5	759.602	0.0000	759.602	0.0070	759.602	0.0070	759.602	0.0000
mpdpsl30-10c	30	4	2	1242.09	0.0000	908.899	0.0010	911.015	0.0010	1242.09	0.0000
mpdpsl30-10d	30	4	5	538.584	0.0000	538.584	0.0010	538.584	0.0000	538.584	0.0000
mpdpsl40-1a	40	2	2	1516.96	0.0010	945.94	0.0270	892.804	0.0270	1516.96	0.0000
mpdpsl40-1b	40	2	5	581.589	0.0040	573.31	0.0780	571.758	0.0760	581.589	0.0000
mpdpsl40-1c	40	4	2	1482.99	0.0000	1003.39	0.0020	1004.93	0.0010	1482.99	0.0000
mpdpsl40-1d	40	4	5	758.756	0.0000	758.756	0.0070	758.756	0.0070	758.756	0.0000
mpdpsl40-2a	40	2	2	1217.33	0.1640	769.459	5.0170	793.984	5.1490	1217.33	0.0010
mpdpsl40-2b	40	2	5	976.293	0.0170	777.785	0.4060	705.77	0.3920	976.293	0.0010
mpdpsl40-2c	40	4	2	1086.92	0.0010	804.385	0.0080	819.121	0.0090	1086.92	0.0000
mpdpsl40-2d	40	4	5	572.944	0.0000	532.073	0.0090	524.3	0.0080	572.944	0.0000
mpdpsl40-3a	40	2	2	1154.9	0.0040	768.607	0.1010	745.923	0.1040	1154.9	0.0010
mpdpsl40-3b	40	2	5	854.125	0.0180	643.547	0.4100	602.697	0.4050	854.125	0.0010
mpdpsl40-3c	40	4	2	1105.05	0.0000	813.182	0.0010	800.215	0.0010	1105.05	0.0000
mpdpsl40-3d	40	4	5	797.569	0.0000	797.569	0.0010	797.569	0.0010	797.569	0.0000
mpdpsl40-4a	40	2	2	1304.66	0.0040	797.21	0.0990	834.494	0.1040	1304.66	0.0000
mpdpsl40-4b	40	2	5	585.428	0.0080	585.933	0.1790	583.982	0.1690	585.428	0.0000
mpdpsl40-4c	40	4	2	1169.57	0.0000	742.758	0.0090	736.702	0.0080	1169.57	0.0000
mpdpsl40-4d	40	4	5	662.14	0.0020	661.685	0.0350	661.485	0.0340	662.14	0.0000
mpdpsl40-5a	40	2	2	1094.51	0.0040	718.746	0.1090	728.368	0.1120	1094.51	0.0000
mpdpsl40-5b	40	2	5	636.513	0.0020	620.69	0.0220	605.634	0.0220	636.513	0.0010
mpdpsl40-5c	40	4	2	634.268	0.0010	475.941	0.0050	459.407	0.0050	634.268	0.0000
mpdpsl40-5d	40	4	5	674.293	0.0000	674.293	0.0010	674.293	0.0010	674.293	0.0000
mpdpsl40-6a	40	2	2	1140.37	0.0040	734.064	0.0910	719.925	0.0940	1140.37	0.0000
mpdpsl40-6b	40	2	5	557.336	0.3170	553.326	8.3750	530.125	7.8060	557.336	0.0000
mpdpsl40-6c	40	4	2	1314.18	0.0000	947.021	0.0050	922.616	0.0050	1314.18	0.0000
mpdpsl40-6d	40	4	5	882.624	0.0010	876.505	0.0040	876.505	0.0040	882.624	0.0000
mpdpsl40-7a	40	2	2	1408.32	0.3400	906.537	10.6260	924.996	10.9780	1408.32	0.0000
mpdpsl40-7b	40	2	5	755.227	0.0040	723.504	0.0800	700.848	0.0760	755.227	0.0010
mpdpsl40-7c	40	4	2	1396.12	0.0080	885.072	0.2140	851.626	0.2140	1396.12	0.0000
mpdpsl40-7d	40	4	5	884.456	0.0000	884.456	0.0020	884.456	0.0010	884.456	0.0000
mpdpsl40-8a	40	2	2	1375.75	0.0020	848.76	0.0470	819.132	0.0470	1375.75	0.0000
mpdpsl40-8b	40	2	5	595.683	0.0010	580.702	0.0380	580.544	0.0370	595.683	0.0000
mpdpsl40-8c	40	4	2	1322.66	0.0090	907.38	0.2170	989.226	0.2220	1322.66	0.0010
mpdpsl40-8d	40	4	5	680.386	0.0000	680.386	0.0010	680.386	0.0010	680.386	0.0010
mpdpsl40-9a	40	2	2	1247.29	0.0030	881.908	0.0450	884.918	0.0460	1247.29	0.0000
mpdpsl40-9b	40	2	5	590.49	0.3110	567.683	7.7280	558.193	7.4670	590.49	0.0010
mpdpsl40-9c	40	4	2	1733.39	0.0020	1082.44	0.0410	1127.56	0.0400	1733.39	0.0000
mpdpsl40-9d	40	4	5	1013.31	0.0010	988.528	0.0070	989.524	0.0070	1013.31	0.0000
mpdpsl40-10a	40	2	2	1335.14	0.0020	805.059	0.0460	820.285	0.0480	1335.14	0.0000
mpdpsl40-10b	40	2	5	992.398	0.1550	741.921	3.9370	695.781	3.7970	992.398	0.0010
mpdpsl40-10c	40	4	2	1243.82	0.0000	785.346	0.0020	785.561	0.0020	1243.82	0.0000
mpdpsl40-10d	40	4	5	489.579	0.0000	489.579	0.0030	489.579	0.0030	489.579	0.0000
mpdpsl50-1a	50	2	2	1776.24	0.1650	1079.97	4.7470	1143.5	4.9490	1776.24	0.0010
mpdpsl50-1b	50	2	5	789.114	0.0360	619.924	0.9020	615.596	0.8960	789.114	0.0010
mpdpsl50-1c	50	4	2	1373.45	0.0000	919.606	0.0060	984.677	0.0050	1373.45	0.0000
mpdpsl50-1d	50	4	5	887.86	0.0010	882.561	0.0200	877.787	0.0190	887.86	0.0010
mpdpsl50-2a	50	2	2	1486.3	0.0380	870.811	1.1100	983.139	1.1980	1486.3	0.0010
mpdpsl50-2b	50	2	5	936.545	0.0360	847.414	0.8520	802.138	0.8050	936.545	0.0000
mpdpsl50-2c	50	4	2	1371.36	0.0040	938.563	0.1020	925.605	0.1010	1371.36	0.0010
mpdpsl50-2d	50	4	5	1485.31	0.3160	1180.18	8.3300	1117.63	7.8860	1485.31	0.0010
mpdpsl50-3a	50	2	2	1752.83	0.7090	1158.37	22.6620	1120.03	23.8290	1752.83	0.0010
mpdpsl50-3b	50	2	5	1193.06	0.6670	928.582	18.4900	955.908	17.7180	1193.06	0.0010
mpdpsl50-3c	50	4	2	1800.66	0.0010	1127.89	0.0190	1153.71	0.0190	1800.66	0.0000
mpdpsl50-3d	50	4	5	983.64	0.0020	984.228	0.0340	978.137	0.0340	983.64	0.0000
mpdpsl50-4a	50	2	2	1767.44	0.0370	1133.5	0.9360	1213.14	0.9290	1767.44	0.0000
mpdpsl50-4b	50	2	5	1047	12.5400	778.137	364.6710	740.808	360.4710	1047	0.0010
mpdpsl50-4c	50	4	2	1585.82	0.0010	1097.29	0.0190	1117.29	0.0190	1585.82	0.0000
mpdpsl50-4d	50	4	5	1060.51	0.0010	1047.98	0.0160	1046.12	0.0170	1060.51	0.0000
mpdpsl50-5a	50	2	2	1687.44	1.4700	1017.83	48.4680	944.935	49.4470	1687.44	0.0010
mpdpsl50-5b	50	2	5	800.606	2.7520	705.041	75.0800	684.404	72.4470	800.606	0.0010
mpdpsl50-5c	50	4	2	1232.47	0.0010	965.883	0.0210	921.43	0.0210	1232.47	0.0000
mpdpsl50-5d	50	4	5	786.655	0.0010	778.963	0.0170	778.128	0.0160	786.655	0.0000
mpdpsl50-6a	50	2	2	1260.43	6.3840	885.799	241.1010	942.193	263.9770	1260.43	0.0020
mpdpsl50-6b	50	2	5	1038.37	12.2410	853.741	360.0820	795.841	23367.0000	1038.37	0.0010
mpdpsl50-6c	50	4	2	1763.51	0.0000	1141.87	0.0060	1094.6	0.0050	1763.51	0.0000
mpdpsl50-6d	50	4	5	848.433	0.0020	831.613	0.0340	827.767	0.0350	848.433	0.0010
mpdpsl50-7a	50	2	2	1560.48	0.0390	1022.48	1.1390	1127.83	1.2090	1560.48	0.0010
mpdpsl50-7b	50	2	5	696.668	0.0090	694.744	0.1970	694.525	0.1920	696.668	0.0010
mpdpsl50-7c	50	4	2	1285.13	0.0010	988.301	0.0070	920.7	0.0080	1285.13	0.0010

Table A.5
(*Continued*).

Instance				Strategy A		Strategy B		Strategy C		Strategy A with theorem	
Name	N	D	Q	E	Time	E	Time	E	Time	E	Time
mpdpsl50-7d	50	4	5	733.932	0.0080	731.988	0.1650	728.647	0.1610	733.932	0.0000
mpdpsl50-8a	50	2	2	1672.94	13.9800	1067.51	532.1680	1058.61	565.9870	1672.94	0.0020
mpdpsl50-8b	50	2	5	881.548	0.0170	849.746	0.3900	841.007	0.3840	881.548	0.0010
mpdpsl50-8c	50	4	2	1608.89	0.0090	1073.17	0.2310	1049.54	0.2350	1608.89	0.0010
mpdpsl50-8d	50	4	5	889.901	0.0010	889.723	0.0100	889.544	0.0100	889.901	0.0000
mpdpsl50-9a	50	2	2	1711.01	0.0210	1059.24	0.5160	1124.57	0.5360	1711.01	0.0010
mpdpsl50-9b	50	2	5	1608.4	1.5300	1013.54	42.9720	1375.11	44.9240	1608.4	0.0030
mpdpsl50-9c	50	4	2	1452.91	0.0000	1033	0.0060	1028.98	0.0070	1452.91	0.0000
mpdpsl50-9d	50	4	5	1109.85	0.0000	1104.17	0.0030	1104.1	0.0030	1109.85	0.0000
mpdpsl50-10a	50	2	2	1468.36	0.0410	874.55	1.1120	859.553	1.1450	1468.36	0.0010
mpdpsl50-10b	50	2	5	727.523	0.0080	727.021	0.1630	690.796	0.1570	727.523	0.0000
mpdpsl50-10c	50	4	2	1626.23	0.0000	1156.21	0.0090	1149.21	0.0090	1626.23	0.0000
mpdpsl50-10d	50	4	5	628.48	0.0000	628.417	0.0050	628.417	0.0040	628.48	0.0000

References

Accorsi, L.A., da Silva, A.R., & Mateus, G.R. (2021). A Tabu search for the Steiner multi cycle problem in networks. *Applied Soft Computing*, *104*, 107188.

Adamo, T., Gendreau, M., Ghiani, G., & Guerriero, E. (2024). A review of recent advances in time-dependent vehicle routing. *European Journal of Operational Research*. https://doi.org/10.1016/j.ejor.2024.06.016

Ahmed, M., Seraj, R., & Islam, S.M.S. (2020). The k-means algorithm: A comprehensive survey and performance evaluation. *Electronics*, *9*(8), 1295. https://doi.org/10.3390/electronics9081295

Akhmetbek, Y. (2023). Stochastic dynamic vehicle routing problem survey. *Suleyman Demirel University Bulletin Natural and Technical Sciences*, *62*(1), 75–87.

Anily, S., & Bramel, J. (1999). Approximation algorithms for the capacitated traveling salesman problem with pickups and deliveries. *Naval Research Logistics (NRL)*, *46*(6), 654–670.

Arishi, A., & Krishnan, K. (2023). A multi-agent deep reinforcement learning approach for solving the multi-depot vehicle routing problem. *Journal of Management Analytics*, *10*(3), 493–515. https://doi.org/10.1080/23270012.2023.2229842

Aydi, H., Chtourou, S., & Frikha, A. (2024). Stochastic pickup and delivery problem: A bibliometric review of research using VOSviewer. In *2024 International conference on decision aid sciences and applications (DASA)* (pp. 1–7). IEEE. https://doi.org/10.1109/DASA63652.2024.10836429

Azarkish, M., & Aghaeipour, Y. (2022). A fuzzy bi-objective mathematical model for multi-depot electric vehicle location routing problem with time windows and simultaneous delivery and pick-up. *Asian Journal of Basic Science and Research*, *4*(2), 01–3.

Azzahra, J.A., & Santosa, B. (2021). Optimization of LPG distribution route using variable neighborhood tabu search algorithm. Ph.D. thesis. Sepuluh Nopember Institute of Technology. https://doi.org/10.12962/j23373539.v10i2.76792

Baghbadorani, R.R., & Haeri, M. (2025). An innovative method to solve classic and dynamic multi-depot vehicle routing problem. *International Journal of Logistics Systems and Management*, *50*(1), 28–50. https://doi.org/10.1504/IJLSM.2025.143890

Bajaj, A., & Dhodiya, J. (2025). Sustainable multi-depot multiple travelling salesman problem by robust multi objectivequasi oppositional jaya algorithm: A case study of surat city. *Journal of Industrial and Management Optimization*, (pp. 2885–2924). https://doi.org/10.3934/jimo.2024198

Barbato, M., Canas, F., Gouveia, L., & Pesneau, P. (2023). Node based compact formulations for the hamiltonian p-median problem. *Networks*, *82*(4), 336–370. https://doi.org/10.1002/net.22163

Barbato, M., & Gouveia, L. (2024). The hamiltonian p-median problem: Polyhedral results and branch-and-cut algorithms. *European Journal of Operational Research*, *316*(2), 473–487. https://doi.org/10.1016/j.ejor.2024.02.032

Bektas, T. (2006). The multiple traveling salesman problem: An overview of formulations and solution procedures. *Omega*, *34*(3), 209–219. https://doi.org/10.1016/j.omega.2004.10.004

Berger, J., Salois, M., & Begin, R. (1998). A hybrid genetic algorithm for the vehicle routing problem with time windows. In *Advances in artificial intelligence: 12th Biennial conference of the canadian society for computational studies of intelligence, AI'98 Vancouver, BC, Canada, June 18–20, 1998 proceedings 12* (pp. 114–127). Springer. https://doi.org/10.1007/3-540-64575-6_44

Bertsimas, D.J. (1992). A vehicle routing problem with stochastic demand. *Operations Research*, *40*(3), 574–585. https://doi.org/10.1287/opre.40.3.574

Bongiovanni, C., Geroliminis, N., & Kaspi, M. (2024). A ride time-oriented scheduling algorithm for dial-a-ride problems. *Computers & Operations Research*, *165*, 106588. https://doi.org/10.1016/j.cor.2024.106588

Boz, E., Calik, A., & Yusuf, Ş. (2025). Solution approaches for the green vehicle routing problem with time window and simultaneous pickup and delivery. https://doi.org/10.17341/gazimmfd.1180965

Branco, I.M., & Coelho, J.D. (1990). The hamiltonian p-median problem. *European Journal of Operational Research*, *47*(1), 86–95. https://doi.org/10.1016/0377-2217(90)90092-p

Carrabs, F., Cordeau, J.F., & Laporte, G. (2007). Variable neighborhood search for the pickup and delivery traveling salesman problem with LIFO loading. *INFORMS Journal on Computing*, *19*(4), 618–632. https://doi.org/10.1287/ijoc.1060.0202

Castro, M.P., Cire, A.A., & Beck, J.C. (2020). An MDD-based lagrangian approach to the multicommodity pickup-and-delivery TSP. *INFORMS Journal on Computing*, *32*(2), 263–278. https://doi.org/10.1287/ijoc.2018.0881

Cornejo-Acosta, J.A., García-Díaz, J., Pérez-Sansalvador, J.C., & Segura, C. (2023). Compact integer programs for depot-free multiple traveling salesperson problems. *Mathematics*, *11*(13), 3014. https://doi.org/10.3390/math11133014

Dang, D.C., & Moukrim, A. (2020). A hybrid VNS for single commodity pickup-and-delivery TSP with loading constraints. *Annals of Operations Research*, *287*(2), 371–395.

Dantzig, G.B., & Ramser, J.H. (1959). The truck dispatching problem. *Management Science*, *6*(1), 80–91. https://doi.org/10.1287/mnsc.6.1.80

Dayarian, I., Savelsbergh, M., & Woensel, T. (2020). An adaptive large neighborhood search for the pickup and delivery problem with handling costs. *Computers & Operations Research*, *115*, 104831.

Dinler, D., & Tural, M.K. (2016). A survey of constrained clustering. In *Unsupervised learning algorithms* (pp. 207–235). Springer. https://doi.org/10.1007/978-3-319-24211-8_9

Dolan, E.D., & Moré, J.J. (2002). Benchmarking optimization software with performance profiles. *Mathematical Programming*, *91*(2), 201–213.

Domínguez-Martín, B., Hernández-Pérez, H., Riera-Ledesma, J., & Rodríguez-Martín, I. (2024). A branch-and-cut algorithm for the one-commodity pickup and delivery location routing problem. *Computers & Operations Research*, *161*, 106426. https://doi.org/10.1016/j.cor.2023.106426

Dridi, I.H., Alaïa, E.B., Borne, P., & Bouchriha, H. (2019). Optimisation of the multi-depots pick-up and delivery problems with time windows and multi-vehicles using PSO algorithm. *International Journal of Production Research*, (pp. 1–14). https://doi.org/10.1080/00207543.2019.1650975

Dubey, N., & Tanksale, A. (2023). A multi-depot vehicle routing problem with time windows, split pickup and split delivery for surplus food recovery and redistribution. *Expert Systems with Applications*, *232*, 120807. https://doi.org/10.1016/j.eswa.2023.120807

Ester, M., Kriegel, H.P., Sander, J., Xu, X. et al. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *KDD* (pp. 226–231). (vol. *96*).

Fernandes, C.G., Lintzmayer, C.N., & Moura, P. F.S. (2024). Approximations for the steiner multicycle problem. *Theoretical Computer Science*, *1020*, 114836. https://doi.org/10.1016/j.tcs.2024.114836

Ge, J., Liu, X., & Liang, G. (2020). Research on vehicle routing problem with soft time windows based on hybrid tabu search and scatter search algorithm. *Computers, Materials & Continua*, *64*(3), 1945–1958. https://doi.org/10.32604/cmc.2020.010977

Gendreau, M., Jabali, O., & Rei, W. (2014). Chapter 8: Stochastic vehicle routing problems. In *Vehicle routing* (pp. 213–239). Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9781611973594.ch8

Gendreau, M., Laporte, G., & Séguin, R. (1996). A tabu search heuristic for the vehicle routing problem with stochastic demands and customers. *Operations Research*, *44*(3), 469–477. https://doi.org/10.1287/opre.44.3.469

Ghorpade, T., & Corlu, C.G. (2023). A simheuristic algorithm for the stochastic one-commodity pickup and delivery travelling salesman problem. *Journal of Simulation*, *17*(6), 688–708. https://doi.org/10.1080/17477778.2022.2062261

Goldberg, D.E., & Lingle, R. (2014). Alleles, loci, and the traveling salesman problem. In *Proceedings of the first international conference on genetic algorithms and their applications* (pp. 154–159). Psychology Press.

Grimm, J. (2025a). The dial-a-ride problem. In *Dial-a-ride problems in transportation service* (pp. 33–73). Springer.

Grimm, J. (2025b). Profits in the dial-a-ride problem. In *Dial-a-ride problems in transportation service* (pp. 75–112). Springer. https://doi.org/10.1007/978-3-031-66346-8_4

Gutin, G., & Punnen, A.P. (2006). The traveling salesman problem and its variations (vol. 12). Springer Science & Business Media.

Haddad, M.N., Martinelli, R., Vidal, T., Martins, S., Ochi, L.S., Souza, M.J.F., & Hartl, R. (2018). Large neighborhood-based metaheuristic and branch-and-price for the pickup and delivery problem with split loads. *European Journal of Operational Research*, *270*(3), 1014–1027. https://doi.org/10.1016/j.ejor.2018.04.017

He, P., Hao, J.K., & Wu, Q. (2024). A hybrid genetic algorithm for the hamiltonian p-median problem. *Networks*, *83*(2), 348–367. https://doi.org/10.1002/net.22197

Hemmelmayr, V.C., Cordeau, J.F., & Crainic, T.G. (2022). An evolutionary framework for dynamic pickup and delivery routing problems. *European Journal of Operational Research*, *301*(2), 739–756.

Hernández-Pérez, H., Landete, M., & Rodríguez-Martín, I. (2021). The single-vehicle two-echelon one-commodity pickup and delivery problem. *Computers & Operations Research*, *127*, 105152. https://doi.org/10.1016/j.cor.2020.105152

Hernández-Pérez, H., & Salazar-González, J.J. (2004). A branch-and-cut algorithm for a traveling salesman problem with pickup and delivery. *Discrete Applied Mathematics*, *145*(1), 126–139. https://doi.org/10.1016/j.dam.2003.09.013

Hernández-Pérez, H., & Salazar-González, J.J. (2022). A branch-and-cut algorithm for the split-demand one-commodity pickup-and-delivery travelling salesman problem. *European Journal of Operational Research*, *297*(2), 467–483. https://doi.org/10.1016/j.ejor.2021.05.040

Ibrahim, M.F., Putri, M.M., Farista, D., & Utama, D.M. (2021). An improved genetic algorithm for vehicle routing problem pick-up and delivery with time windows. *Jurnal Teknik Industri*, *22*(1), 1–17.

Ibrahim, S., & Jarboui, B. (2022). A general variable neighborhood search approach based on a p-median model for cellular manufacturing problems. *Optimization Letters*, *16*(1), 137–151. https://doi.org/10.1007/s11590-020-01662-4

Indravattana, T., & Silawan, T. (2023). Load balancing in multi depot pickup and delivery problem with ant colony optimization. In *2023 8th International conference on business and industrial research (ICBIR)* (pp. 913–916). IEEE. https://doi.org/10.1109/ICBIR57571.2023.10147562

Jiang, J., Dai, Y., Yang, F., & Ma, Z. (2024). A multi-visit flexible-docking vehicle routing problem with drones for simultaneous pickup and delivery services. *European Journal of Operational Research*, *312*(1), 125–137. https://doi.org/10.1016/j.ejor.2023.06.021

Jodeau, J., Absi, N., Chevrier, R., & Feillet, D. (2024). The rail-road dial-a-ride problem. *European Journal of Operational Research*, *318*(2), 486–499. https://doi.org/10.1016/j.ejor.2024.05.036

Johnsen, L.C., Meisel, F., & Ehmke, J.F. (2025). Stochastic stay times for interrelated trips in the rural dial-a-ride problem. *Transportation Research Part E: Logistics and Transportation Review*, *195*, 103968. https://doi.org/10.1016/j.tre.2025.103968

Konstantakopoulos, G.D., Gayialis, S.P., & Kechagias, E.P. (2022). Vehicle routing problem and related algorithms for logistics distribution: A literature review and classification. *Operational Research*, *22*(3), 2033–2062. https://doi.org/10.1007/s12351-020-00600-7

Kuo, Y., & Wang, C.C. (2012). A variable neighborhood search for the multi-depot vehicle routing problem with loading cost. *Expert Systems with Applications*, *39*(8), 6949–6954. https://doi.org/10.1016/j.eswa.2012.01.024

Kyriakakis, N.A., Sevastopoulos, I., Marinaki, M., & Marinakis, Y. (2022). A hybrid tabu search–variable neighborhood descent algorithm for the cumulative capacitated vehicle routing problem with time windows in humanitarian applications. *Computers & Industrial Engineering*, *164*, 107868. https://doi.org/10.1016/j.cie.2021.107868

Labidi, H., Azzouna, N.B., Hassine, K., & Gouider, M.S. (2023). An improved genetic algorithm for solving the multi-objective vehicle routing problem with environmental considerations. *Procedia Computer Science*, *225*, 3866–3875. https://doi.org/10.1016/j.procs.2023.10.382

Lehmann, J., & Winkenbach, M. (2024). A matheuristic for the two-echelon multi-trip vehicle routing problem with mixed pickup and delivery demand and time windows. *Transportation Research Part C: Emerging Technologies*, *160*, 104522. https://doi.org/10.1016/j.trc.2024.104522

Li, K., Zhang, L., & Zhang, T. (2020). Improved ACO for multi-depot pickup and delivery with hard time windows. *Computers & Operations Research*, *122*, 104988. https://doi.org/10.1007/978-981-10-1837-4_96

Li, Q., Bai, R., & Ouelhadj, D. (2022). Adaptive ant colony optimization for multi-depot, multi-vehicle pickup and delivery problems with time windows. *European Journal of Operational Research*, *301*(1), 120–138.

Li, R., Yuan, Q., Liao, T., Luo, L., Hu, M., Lai, P., & Zhang, X. (2024). Dynamic task scheduling for coordinated truck-drone parcel delivery: A hybrid genetic tabu algorithm approach. In *2024 IEEE International symposium on parallel and distributed processing with applications (ISPA)* (pp. 940–947). IEEE. https://doi.org/10.1109/ISPA63168.2024.00125

Lintzmayer, C.N., Miyazawa, F.K., Moura, P. F.S., & Xavier, E.C. (2020). Randomized approximation scheme for steiner multi cycle in the euclidean plane. *Theoretical Computer Science*, *835*, 134–155. https://doi.org/10.1016/j.tcs.2020.06.022

Liu, C., Quilliot, A., Toussaint, H., & Feillet, D. (2024a). A filtering system to solve the large-scale shared autonomous vehicles dial-a-ride problem. *Transportation Research Part C: Emerging Technologies*, *161*, 104551. https://doi.org/10.1016/j.trc.2024.104551

Liu, W., Qiu, J., Deng, J., Zheng, N., Chang, X., & Liu, Y. (2024b). Variable neighbourhood search embedded perturbation mechanism for multi-depot vehicle routing problem with simultaneous delivery & pickup, and time limit. *Computers & Industrial Engineering*, *189*, 109942. https://doi.org/10.1016/j.cie.2024.109942

López-Ibáñez, M., Dubois-Lacoste, J., Cáceres, L.P., Birattari, M., & Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, *3*, 43–58. https://doi.org/10.1016/j.orp.2016.09.002

Lou, P., Xu, K., Yan, J., & Xiao, Z. (2021). An improved partheno-genetic algorithm for open path multi-depot multiple traveling salesmen problem. In *Journal of Physics: Conference Series* (p. 012002). IOP Publishing (*vol. 1848*). https://doi.org/10.1088/1742-6596/1848/1/012002

Lv, H., Liu, R., & Li, J. (2025). A multitasking ant system for multi-depot pick-up and delivery location routing problem with time window. *Complex & Intelligent Systems*, *11*(2), 1–21. https://doi.org/10.1007/s40747-024-01750-3

Ma, J., Zhang, J., & Guo, Y. (2022). The stochastic and dynamic vehicle routing problem: A literature review. In *Proceedings of 2021 Chinese intelligent automation conference* (pp. 344–351). Springer. https://doi.org/10.1007/978-981-16-6372-7_39

Mardešić, N., Erdelić, T., Carić, T., & Đurasević, M. (2024). Review of stochastic dynamic vehicle routing in the evolving urban logistics environment. *Mathematics*, *12*(1). https://doi.org/10.3390/math12010028

Mladenović, N., & Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research*, *24*(11), 1097–1100. https://doi.org/10.1016/s0305-0548(97)00031-2

Murtagh, F., & Contreras, P. (2012). Algorithms for hierarchical clustering: An overview. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, *2*(1), 86–97. https://doi.org/10.1002/widm.53

Nagata, Y., & Kobayashi, S. (2013). A powerful genetic algorithm using edge assembly crossover for the traveling salesman problem. *INFORMS Journal on Computing*, *25*(2), 346–363. https://doi.org/10.1287/ijoc.1120.0506

Ng, X.J., Kerk, Y.W., Lim, T.Y., & Tan, C.J. (2024). Metaheuristic tabu search for vehicle scheduling: A case study of healthcare logistics. In *Advances in intelligent healthcare delivery and management: Research papers in honour of professor maria virvou for invaluable contributions* (pp. 221–236). Springer. https://doi.org/10.1007/978-3-031-65430-5_11

de Novaes Guimarães, P.V., San Felice, M.C., Hokama, P.H.D.B., & Xavier, E.C. (2024). On the restricted steiner multi cycle problem. *Computers & Operations Research*, *169*, 106752. https://doi.org/10.1016/j.cor.2024.106752

Ombuki, B., Ross, B.J., & Hanshar, F. (2006). Multi-objective genetic algorithms for vehicle routing problem with time windows. *Applied Intelligence*, *24*, 17–30. https://doi.org/10.1007/s10489-006-6926-z

Ousmane, B., Moustapha, D., & Adama, C. (2021). Genetic algorithm for the pick-up and delivery problem with time window by multi-compartment vehicles. *Transport and Telecommunication*, *22*(3), 343–352. https://doi.org/10.2478/ttj-2021-0027

Oyola, J., Arntzen, H., & Woodruff, D.L. (2016a). The stochastic vehicle routing problem, a literature review, part I: Models. *EURO Journal on Transportation and Logistics*, *7*(3), 193–221. https://doi.org/10.1007/s13676-016-0100-5

Oyola, J., Arntzen, H., & Woodruff, D.L. (2016b). The stochastic vehicle routing problem, a literature review, part II: Solution methods. *EURO Journal on Transportation and Logistics*, *6*(4), 349–388. https://doi.org/10.1007/s13676-016-0099-7

Öztaş, T., & Tuş, A. (2022). A hybrid metaheuristic algorithm based on iterated local search for vehicle routing problem with simultaneous pickup and delivery. *Expert Systems with Applications*, *202*, 117401. https://doi.org/10.1016/j.eswa.2022.117401

Palacio, J.D., & Rivera, J.C. (2022). A multi-start evolutionary local search for the one-commodity pickup and delivery traveling salesman problem. *Annals of Operations Research*, *316*(2), 979–1011. https://doi.org/10.1007/s10479-020-03789-0

Pan, Y., & Xia, Y. (2014). Solving TSP by dismantling cross paths. In *2014 International conference on orange technologies*. IEEE. https://doi.org/10.1109/icot.2014.6956614

Park, H., Son, D., Koo, B., & Jeong, B. (2021). Waiting strategy for the vehicle routing problem with simultaneous pickup and delivery using genetic algorithm. *Expert Systems with Applications*, *165*, 113959. https://doi.org/10.1016/j.eswa.2020.113959

Pereira, V.N.G., San Felice, M.C., Hokama, P.H.D.B., & Xavier, E.C. (2018). The steiner multi cycle problem with applications to a collaborative truckload problem. In *17th International symposium on experimental algorithms (SEA 2018)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik. https://doi.org/10.4230/LIPICS.SEA.2018.26

Perez-Carabaza, S., Galvez, A., & Iglesias, A. (2024). Ant colony based dynamic voronoi method for the multi-depot multiple TSP. In *2024 IEEE Congress on evolutionary computation (CEC)* (pp. 1–8). IEEE. https://doi.org/10.1109/CEC60901.2024.10612048

Phuc, P.N.K., & Thao, N.L.P. (2021). Ant colony optimization for multiple pickup and multiple delivery vehicle routing problem with time window and heterogeneous fleets. *Logistics*, *5*(2), 1–13. https://doi.org/10.3390/logistics5020028

Qasimi, M.A. (2024). Solving the travelling salesman problem using the PSO optimization. *TechComp Innovations: Journal of Computer Science and Technology*, *1*(1), 19–27. https://doi.org/10.70063/techcompinnovations.v1i1.20

Rajesh, K., Jain, E., & Kotecha, P. (2022). A multi-objective approach to the electric vehicle routing problem. arXiv preprint arXiv:2208.12440 https://doi.org/10.48550/arXiv.2208.12440

Ramanathan, T., Suresh, S., & Rao, T.S. (2023). Multiple depot MTSP using genetic algorithm and reinforcement learning. In *2023 Second international conference on augmented intelligence and sustainable systems (ICAISS)* (pp. 440–446). IEEE. https://doi.org/10.1109/ICAISS58487.2023.10250669

Raza, S.M., Sajid, M., & Singh, J. (2022). Vehicle routing problem using reinforcement learning: Recent advancements. In *Advanced machine intelligence and signal processing* (pp. 269–280). Springer. https://doi.org/10.1007/978-981-19-0840-8_20

Research and Markets (2024). Online food delivery services global market report 2024. Accessed: 2025-02-25. https://www.researchandmarkets.com/reports/5315009/online-food-delivery-services-global-market?utm_source=BW&utm_medium=PressRelease&utm_code=2clw6w&utm_campaign=1531597+-+Global+Online+Food+Delivery+Services+Market+Report+2021%3a+COVID-19+Growth%2c+Impacts+and+Change+to+2030&utm_exec=chdo54prd.

Rios, B.H.O., Xavier, E.C., Miyazawa, F K., & Amorim, P. (2020). Stochastic multi-depot vehicle routing problem with pickup and delivery: an ILS approach. In *Proceedings of the 2020 federated conference on computer science and information systems*. IEEE. https://doi.org/10.15439/2020f127

Rios, B.H.O., Xavier, E.C., Miyazawa, F.K., Amorim, P., Curcio, E., & Santos, M.J. (2021). Recent dynamic vehicle routing problems: A survey. *Computers & Industrial Engineering*, *160*, 107604. https://doi.org/10.1016/j.cie.2021.107604

Rist, Y., & Forbes, M.A. (2021). A new formulation for the dial-a-ride problem. *Transportation Science*, *55*(5), 1113–1135. https://doi.org/10.1287/trsc.2021.1044

Ropke, S., & Santini, A. (2022). Adaptive large neighborhood search for the one-commodity pickup-and-delivery traveling salesman problem. *Computers & Operations Research*, *142*, 105741. https://doi.org/10.1007/s10479-020-03789-0

Sadati, M.E.H., & Çatay, B. (2021). A hybrid variable neighborhood search approach for the multi-depot green vehicle routing problem. *Transportation Research Part E: Logistics and Transportation Review*, *149*, 102293. https://doi.org/10.1016/j.tre.2021.102293

Sadati, M.E.H., Çatay, B., & Aksen, D. (2021). An efficient variable neighborhood search with tabu shaking for a class of multi-depot vehicle routing problems. *Computers & Operations Research*, *133*, 105269. https://doi.org/10.1016/j.cor.2021.105269

Sahoo, S., Cordeau, J.F., & Laporte, G. (2020). Metaheuristics for a combined Hamiltonian p-median and route design problem. *Computers & Industrial Engineering*, *147*, 106677.

Schenekemberg, C.M., Chaves, A.A., Coelho, L.C., Guimarães, T.A., & Avelino, G.G. (2022). The dial-a-ride problem with private fleet and common carrier. *Computers & Operations Research*, *147*, 105933. https://doi.org/10.1016/j.cor.2022.105933

Soeffker, N., Ulmer, M.W., & Mattfeld, D.C. (2022). Stochastic dynamic vehicle routing in the light of prescriptive analytics: A review. *European Journal of Operational Research*, *298*(3), 801–820. https://doi.org/10.1016/j.ejor.2021.07.014

Su, Y., Dupin, N., & Puchinger, J. (2023). A deterministic annealing local search for the electric autonomous dial-a-ride problem. *European Journal of Operational Research*, *309*(3), 1091–1111. https://doi.org/10.1016/j.ejor.2023.02.012

Subramanian, A., Drummond, L.M.A., Bentes, C., Ochi, L.S., & Farias, R. (2010). A parallel heuristic for the vehicle routing problem with simultaneous pickup and delivery. *Computers & Operations Research*, *37*(11), 1899–1911. https://doi.org/10.1016/j.cor.2009.10.011

Sun, K., Zheng, D., Song, H., Cheng, Z., Lang, X., Yuan, W., & Wang, J. (2023). Hybrid genetic algorithm with variable neighborhood search for flexible job shop scheduling problem in a machining system. *Expert Systems with Applications*, *215*, 119359. https://doi.org/10.1016/j.eswa.2022.119359

Tan, S.Y., & Yeh, W.C. (2021). The vehicle routing problem: State-of-the-art classification and review. *Applied Sciences*, *11*(21), 10295. https://doi.org/10.1016/j.cie.2015.12.007

Torres-Tapia, W., Montoya-Torres, J., & Ruiz-Meza, J. (2022). Hybrid ils-vnd algorithm for the vehicle routing problem with release times. In *Workshop on engineering applications* (pp. 222–233). Springer. https://doi.org/10.1007/978-3-031-20611-5_19

Ulmer, M.W., Goodson, J.C., Mattfeld, D.C., & Thomas, B.W. (2020). On modeling stochastic dynamic vehicle routing problems. *EURO Journal on Transportation and Logistics*, *9*(2), 100008. https://doi.org/10.1016/j.ejtl.2020.100008

Vidal, T., Crainic, T.G., Gendreau, M., & Prins, C. (2014). A unified solution framework for multi-attribute vehicle routing problems. *European Journal of Operational Research*, *234*(3), 658–673. https://doi.org/10.1016/j.ejor.2013.09.045

Vidal, T., Laporte, G., & Matl, P. (2020). A concise guide to existing and emerging vehicle routing problem variants. *European Journal of Operational Research*, *286*(2), 401–416. https://doi.org/10.1016/j.ejor.2019.10.010

Vieira, Y.E.M., de Mello Bandeira, R.A., & da Silva Júnior, O.S. (2021). Multi-depot vehicle routing problem for large scale disaster relief in drought scenarios: The case of the brazilian northeast region. *International Journal of Disaster Risk Reduction*, *58*, 102193. https://doi.org/10.1016/j.ijdrr.2021.102193

Wang, L., Ding, Y., Chen, Z., Su, Z., & Zhuang, Y. (2024). Heuristic algorithms for heterogeneous and multi-trip electric vehicle routing problem with pickup and delivery. *World Electric Vehicle Journal*, *15*(2), 69. https://doi.org/10.3390/wevj15020069

Wei, X., Niu, C., Zhao, L., & Wang, Y. (2025). Combination of ant colony and student psychology based optimization for the multi-depot electric vehicle routing problem with time windows. *Cluster Computing*, *28*(2), 99. https://doi.org/10.1007/s10586-024-04821-9

Weng, X., She, W., Fan, H., Zhang, J., & Yun, L. (2025). Multi-depot vehicle routing problem with drones in emergency logistics. *Cluster Computing*, *28*(1), 1–27. https://doi.org/10.1007/s10586-024-04809-5

Wu, H., & Gao, Y. (2023). An ant colony optimization based on local search for the vehicle routing problem with simultaneous pickup–delivery and time window. *Applied Soft Computing*, *139*, 110203. https://doi.org/10.1016/j.asoc.2023.110203

Xu, G., Xiang, T., Li, Y., & Li, J. (2022). A model with an efficient tabu search for the cash pickup and delivery problem. *International Journal of Shipping and Transport Logistics*, *15*(1-2), 71–90. https://doi.org/10.1504/IJSTL.2022.125834

Xu, Y., Tong, Y., Shi, Y., Tao, Q., Xu, K., & Li, W. (2020). An efficient insertion operator in dynamic ridesharing services. *IEEE Transactions on Knowledge and Data Engineering*, *34*(8), 3583–3596. https://doi.org/10.1109/TKDE.2020.3027200

Xue, S. (2023). An adaptive ant colony algorithm for crowdsourcing multi-depot vehicle routing problem with time windows. *Sustainable Operations and Computers*, *4*, 62–75. https://doi.org/10.1016/j.susoc.2023.02.002

Yernar, A., & Turan, C. (2025). Recent developments in vehicle routing problem under time uncertainty: A comprehensive review. *Bulletin of Electrical Engineering and Informatics*, *14*(2), 1263–1275. https://doi.org/10.11591/eei.v14i2.8636

Yi, L., Gao, Y., & Wang, R. (2022). Hierarchical PSO approach for multi-base multiple TSP in smart logistics. *Computers & Industrial Engineering*, *166*, 107989.

Zhang, K., Lin, X., & Li, M. (2023). Graph attention reinforcement learning with flexible matching policies for multi-depot vehicle routing problems. *Physica A: Statistical Mechanics and its Applications*, *611*, 128451. https://doi.org/10.1016/j.physa.2023.128451

Zhang, Z., Che, Y., & Liang, Z. (2024). Split-demand multi-trip vehicle routing problem with simultaneous pickup and delivery in airport baggage transit. *European Journal of Operational Research*, *312*(3), 996–1010. https://doi.org/10.1016/j.ejor.2023.07.028

Zhao, W., Bian, X., & Mei, X. (2024). An adaptive multi-objective genetic algorithm for solving heterogeneous green city vehicle routing problem. *Applied Sciences*, *14*(15), 6594. https://doi.org/10.3390/app14156594